



Universidad de
Oviedo



ESCUELA POLITÉCNICA DE INGENIERÍA DE GIJÓN

GRADO EN CIENCIA E INGENIERÍA DE DATOS

ÁREA DE CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL

MONITORIZACIÓN DE LA PRESENCIA DE ZOSTERA NOLTII EN LA RÍA DE VILLAVICIOSA A PARTIR DE IMÁGENES CAPTURADAS CON DRONES

D. ELVIRA SÁNCHEZ, PELAYO

TUTOR/ES:

**D. GONZÁLEZ GONZÁLEZ, PABLO
D. GARCÍA GONZÁLEZ, FERNANDO**

FECHA: Junio 2025

Índice de contenido

Índice de contenido	2
Índice de figuras	4
Índice de tablas	6
Glosario de abreviaturas y acrónimos	7
1. Introducción	9
2. Hipótesis de partida	11
2.1. Revisión del estado del arte en el monitoreo de praderas marinas	11
2.2. Contexto ecológico y marco normativo	13
3. Objetivos concretos y relación con el estado actual	16
3.1. Objetivos concretos	16
3.2. Estado actual de la segmentación de praderas marinas	17
3.3. Solución propuesta: Segmentación con redes neuronales profundas	18
4. Metodología de trabajo	25
4.1. Datos	25
4.1.1. Imágenes en formato <i>TIFF</i>	25
4.1.2. Máscaras en formato <i>GeoJSON</i>	28
4.2. Herramientas software	32
4.2.1. Sistema de información geográfica	32
4.2.2. Lenguaje de programación y bibliotecas	32
4.2.3. Frameworks	33
4.3. Preprocesamiento de datos	34
4.3.1. Paso de <i>GeoJSON</i> a máscaras binarias en formato ráster	34
4.3.2. Reducción de la resolución	37
4.3.3. Posprocesado de máscaras	39
4.3.4. Obtención y filtrado de <i>patches</i>	40
4.4. Estructura y preparación del flujo de entrenamiento	42
4.4.1. Carga de los datos	42
4.4.2. Flujo de entrenamiento	43
4.5. Implementación del módulo de inferencia	49
5. Experimentación y resultados obtenidos	51
5.1. Especificaciones hardware de la máquina	51
5.2. Arquitectura del modelo	52
5.3. Evaluación del impacto del filtrado de imágenes y máscaras sin regiones de interés	55
5.3.1. Ajuste del modelo con el conjunto de datos original	56
5.3.2. Ajuste del modelo con el conjunto de datos filtrado	60
5.3.3. Discusión de los resultados	64
5.4. Evaluación del rendimiento con ortomosaicos a distintas resoluciones	67

5.4.1.	Ajuste del modelo con ortomosaicos de mayor resolución (0.05 m/píxel)	68
5.4.2.	Ajuste del modelo con ortomosaicos de menor resolución (0.10 m/píxel)	69
5.4.3.	Discusión de los resultados	73
6.	Conclusiones y trabajos futuros	75
6.1.	Conclusiones	75
6.2.	Trabajos futuros	78
6.2.1.	Empleo de una versión refinada de las máscaras	78
6.2.2.	Empleo de imágenes multiespectrales	78
	Bibliografía	80
A.	Anexo I: Ajuste de hiperparámetros	83

Índice de figuras

2.1. Aspecto de una pradera formada por <i>Zostera marina</i> (hojas anchas) y <i>Nanozostera noltii</i> (hojas estrechas). <i>Fotografía Victor M. Vázquez</i> [1].	14
2.2. Medidas de gestión en la zona de la ría de Villaviciosa. En verde, estuarios con plantas marinas; en rayas diagonales de color azul, zona protegida por <i>Natura 2000</i> ; en rosa, zona <i>Ramsar</i> ; puntos amarillo, vertidos urbanos [1].	15
3.1. Diagrama de los diferentes campos de la inteligencia artificial.	19
3.2. Diferencia entre capa totalmente conectada (izquierda) y capa convolucional (derecha) [2].	19
3.3. Arquitectura de <i>FCN</i> presentada en el artículo [3].	21
3.4. Arquitectura <i>U-Net</i> [4].	22
3.5. Arquitectura <i>DeepLabv3+</i> [5].	23
4.1. Ortomosaico de la ría de Villaviciosa correspondiente a la zona de Bornizal.	26
4.2. Segmento del archivo <i>GeoJSON</i> <i>Nanozostera_noltei</i>	28
4.3. Ortomosaico de una parte de la ría de Villaviciosa con su respectiva zona de interés superpuesta.	29
4.4. Representación del mosaico de la ría de Villaviciosa; superpuesto, en color naranja las regiones de interés; en color rosa los puntos de presencia y en azul los puntos de ausencia.	30
4.5. Distribución de las clases presentes en el archivo de <i>groundtruth</i>	31
4.6. Imagen de un sector de la ría de Villaviciosa; superpuesta la máscara en formato <i>GeoJSON</i>	37
4.7. Máscara asociada a la imagen de la ría 4.6	37
4.8. Imagen de un sector de la ría de Villaviciosa.	40
4.9. Máscara sin procesar asociada a la Figura 4.8	40
4.10. Máscara procesada asociada a la imagen de la Figura 4.8	40
5.1. Diagrama del modelo empleado en la experimentación.	52
5.2. Ortomosaico empleado para validación junto con su máscara binaria. Los puntos de <i>ground truth</i> indican presencia (rosa) o ausencia (azul) de <i>N. Noltei</i>	56
5.3. Comparación de la evolución del error y del <i>learning rate</i> durante el entrenamiento	58
5.4. Máscara predicha por el modelo entrenado con el conjunto original. Superpuestos, los puntos de <i>ground truth</i> que indican presencia (rosa) o ausencia (azul)	59
5.5. Matriz de confusión del modelo entrenado a partir del conjunto original	60
5.6. Comparación de la evolución del error y del <i>learning rate</i> durante el entrenamiento con el conjunto de datos filtrado	62
5.7. Máscara predicha por el modelo entrenado con el conjunto filtrado. Superpuestos, los puntos de <i>ground truth</i> que indican presencia (rosa) o ausencia (azul)	63
5.8. Matriz de confusión del modelo entrenado a partir del conjunto filtrado	64
5.9. Comparación de la evolución del error y del <i>learning rate</i> durante el entrenamiento con imágenes a menor resolución	70

5.10. Máscara predicha por el modelo entrenado con las imágenes a menor resolución. Superpuestos, los puntos de <i>ground truth</i> que indican presencia (rosa) o ausencia (azul)	71
5.11. Matriz de confusión del modelo entrenado a partir del conjunto de imágenes con menor resolución	72
A.1. Fluctuación del error de validación durante las distintas iteraciones.	85
A.2. Importancia asignada por Optuna a cada hiperparámetro en relación con su contribución a la reducción del error de validación.	86
A.3. Máscara predicha por el modelo con los mejores hiperparámetros tras la optimización.	87
A.4. Matriz de confusión del modelo con los mejores hiperparámetros tras la optimización.	88

Índice de tablas

4.1. Características de los ortomosaicos originales.	27
4.2. Características de los ortomosaicos tras aplicar el proceso de reducción de resolución.	38
4.3. Distribución de imágenes por conjunto	43
5.1. Hiperparámetros empleados durante el entrenamiento del modelo.	54
5.2. Resultados en el conjunto de test empleando el modelo entrenado con el conjunto de datos original	59
5.3. Distribución de imágenes por conjunto una vez filtrado	61
5.4. Resultados en el conjunto de test empleando el modelo entrenado con el conjunto de datos filtrado	63
5.5. Características de los ortomosaicos tras aplicar la reducción más drástica de resolución (0.1 metros/píxel).	68
5.6. Distribución de imágenes por conjunto una vez filtrado	69
5.7. Resultados en el conjunto de test empleando el modelo entrenado con las imágenes a menor resolución	71
A.1. Rangos de búsqueda de los hiperparámetros	84
A.2. Hiperparámetros óptimos	84

Glosario de abreviaturas y acrónimos

Abreviaturas

N. *noltei* Abreviatura científica de la planta marina *Nanozostera noltei*.

P. *oceanica* Abreviatura científica de la planta marina *Posidonia oceanica*.

MB Megabyte.

GB Gigabyte.

px Pixel.

GiB Gibibyte.

GHz Gigahercios.

Acrónimos

CSIC Consejo Superior de Investigaciones Científicas.

IEO Instituto Español de Oceanografía.

LESPRE Listado de Especies Silvestres en Régimen de Protección Especial.

LIC Lugar de Interés Comunitario.

MITECO Ministerio para la Transición Ecológica y el Reto Demográfico.

ZEPA Zona de Especial Protección para las Aves.

AMV Autonomous Marine Vehicle (Vehículo Marino Autónomo)

DNN Deep Neural Network (Red Neuronal Profunda).

CNN Convolutional Neural Network (Red Neuronal Convolutacional).

FCN Fully Convolutional Network (Red Neuronal Completamente Convolutacional).

ASPP Atrous Spatial Pyramid Pooling (Agrupamiento Piramidal Espacial con Dilatación).

TIFF Tagged Image File Format

CRS Coordinate Reference System (Sistema de Referencia de Coordenadas).

SIG Sistema de Información Geográfica.

JSON JavaScript Object Notation.

RGB Red, Green, Blue (Rojo, Verde, Azul).

WGS World Geodetic System (Sistema Geodésico Mundial).

GPU Graphics Processing Unit (Unidad de Procesamiento Gráfico).

BCE Binary Cross-Entropy.

SSH Secure Shell (Protocolo de acceso remoto seguro).

VRAM Video Random Access Memory (Memoria de acceso aleatorio para video).

LTS Long Term Support (Soporte a largo plazo).

1. Introducción

La conservación de los ecosistemas marinos es una prioridad en la actualidad, y entre ellos, las praderas de *Nanozostera noltei* desempeñan un papel fundamental tanto en la biodiversidad local como en la provisión de servicios ecosistémicos. Este trabajo surge como una colaboración entre el Grupo de Ecología de Angiospermas Marinas del Instituto Español de Oceanografía (IEO-CSIC), a través del Centro Oceanográfico de Gijón, y la Universidad de Oviedo, concretamente con el área de Ciencias de la Computación e Inteligencia Artificial de la Escuela Politécnica de Ingeniería de Gijón. Su objetivo principal es automatizar el proceso de monitorización del estado de las praderas de *N. noltei* a partir de imágenes aéreas capturadas con dron en la ría de Villaviciosa.

El trabajo se enmarca en el uso de técnicas de *machine learning* como vía para abordar una tarea que, de forma manual, resulta altamente costosa y poco escalable. La creciente disponibilidad de imágenes aéreas de alta resolución genera un volumen de datos inabarcable para ser procesado por métodos tradicionales. Frente a esta limitación, se propone una solución basada en redes neuronales profundas, que permita realizar una segmentación automatizada de las zonas de interés. De este modo, se facilita una nueva vía para la monitorización eficiente de estos ecosistemas, aprovechando el potencial del aprendizaje automático para abordar tareas que de otro modo serían inasumibles a gran escala.

A lo largo de este trabajo se abordará de manera integral el procesamiento y análisis de las imágenes aéreas, desde la elaboración de un conjunto de datos lo más robusto posible hasta el entrenamiento y evaluación de un modelo de segmentación binaria. Se explorarán diversas metodologías para optimizar la calidad del conjunto de datos, como la construcción de máscaras binarias a partir de archivos vectoriales en formato *GeoJSON*, su rasterización, y la aplicación de técnicas de filtrado para eliminar ruido e información irrelevante. Además, se evaluará el impacto de trabajar con distintas resoluciones espaciales y configuraciones de los datos de entrada, lo que permitirá analizar la viabilidad y rendimiento del enfoque ante distintos escenarios de monitorización a gran escala.

Los datos utilizados en este trabajo provienen de dos fuentes principales: imágenes ortorectificadas generadas a partir de vuelos con dron, y archivos vectoriales con información geoespacial. A estos se suman los metadatos geográficos incorporados en los propios ortomosaicos, fundamentales para su procesamiento. Los archivos vectoriales comprenden tanto las máscaras utilizadas durante el preprocesamiento como un conjunto de puntos de validación empleados posteriormente en la fase de evaluación. Estas fuentes se integran y procesan en distintas etapas del flujo de trabajo, siendo las máscaras clave para la construcción del con-

junto de datos.

Esta memoria busca presentar los pasos seguidos y los resultados obtenidos, ofreciendo un marco de referencia para la toma de decisiones en futuras investigaciones y aplicaciones en la conservación de praderas marinas. Gracias a este documento se podrá comprender la metodología aplicada y las implicaciones de los hallazgos, demostrando que el uso de técnicas de segmentación automatizada es una herramienta prometedora para mejorar la monitorización y la gestión sostenible de estos ecosistemas.

Todo el código fuente empleado, así como las funciones desarrolladas para cada una de las fases del flujo de trabajo, se encuentra disponible en el repositorio de GitHub asociado a este proyecto.¹

¹<https://github.com/pelayoelvira/zostera>

2. Hipótesis de partida

Este trabajo parte de la hipótesis central de que es posible aplicar técnicas avanzadas de aprendizaje automático, y en particular de *deep learning*, para automatizar la segmentación de imágenes aéreas con la presencia de la planta marina *Nanozostera noltei*. Se plantea que el uso de redes neuronales profundas permitirá detectar patrones espaciales complejos presentes en las imágenes obtenidas con dron, facilitando así la identificación automática y precisa de las zonas ocupadas por esta especie. Este enfoque tiene el potencial de mejorar significativamente la eficiencia y escalabilidad de los procesos de monitorización, que hasta ahora dependen mayoritariamente de métodos manuales, costosos y poco escalables. Al automatizar la segmentación, se abre la posibilidad de llevar a cabo un seguimiento sistemático y a gran escala de las praderas de *N. noltei*, contribuyendo a la conservación y gestión efectiva de estos ecosistemas.

Para fundamentar la hipótesis de este trabajo, es importante destacar que la automatización mediante técnicas de *machine learning* responde no solo a una necesidad práctica, sino que también se apoya en avances recientes en el análisis de imágenes en entornos marinos. Estas técnicas, especialmente el aprendizaje profundo, son capaces de extraer y aprender patrones espaciales complejos en imágenes, facilitando la identificación de especies vegetales en condiciones ambientales variables y fondos heterogéneos. Estas capacidades motivan una revisión detallada de los principales avances en el uso de aprendizaje profundo para el monitoreo automatizado de praderas marinas, la cual se presenta a continuación.

2.1.- Revisión del estado del arte en el monitoreo de praderas marinas

El monitoreo y mapeo de praderas marinas mediante técnicas de visión artificial y aprendizaje profundo ha sido un campo de creciente interés en la última década. En particular, la *Posidonia oceanica* ha sido objeto de numerosos estudios debido a su importancia ecológica y su alarmante retroceso en diversas regiones del Mediterráneo. La combinación de imágenes de alta resolución y redes neuronales profundas ha demostrado ser una herramienta eficiente para la identificación y monitorización de estas praderas.

Uno de los enfoques más destacados en este ámbito es el uso de Vehículos Marinos Autónomos (AMVs, por sus siglas en inglés) equipados con cámaras que toman imágenes sobre el fondo marino. Mediante redes neuronales convolucionales se mapea de manera automática la cobertura del fondo marino [6]. Este tipo de sistemas ha demostrado ser altamente efectivo para generar mapas 2D y 3D de la distribución de *P. oceanica*, facilitando el mo-

nitoreo continuo de estos ecosistemas con menor coste y mayor seguridad en comparación con los métodos tradicionales basados en buzos. Asimismo, se ha explorado la posibilidad de integrar segmentación semántica en *AMVs* para la detección en tiempo real de *P. oceanica*, alcanzando precisiones superiores al 96 % [7].

Existe otro estudio que también emplea *AMVs* para analizar la evolución temporal de las praderas de *P. oceanica*. En este caso, los vehículos siguen trayectorias predefinidas y capturan imágenes del fondo marino de manera sistemática. Posteriormente, las imágenes recopiladas son procesadas para obtener mosaicos, que consisten en la unión de múltiples imágenes individuales para formar representaciones continuas de grandes extensiones del lecho marino. Esta metodología consigue cuantificar la cobertura de *P. oceanica* en distintos periodos e identificar tendencias de expansión o declive en respuesta a factores ambientales y humanos [8].

Además del uso de vehículos autónomos, otro enfoque para la monitorización de praderas marinas se basa en el análisis de imágenes satelitales. Un estudio reciente [9] ha desarrollado un modelo basado en redes neuronales convolucionales para mapear la distribución de *P. oceanica* en el Mar Mediterráneo a partir de datos de teledetección. Utilizando un conjunto de datos georreferenciado de hábitats marinos y una variedad de imágenes satelitales, los autores lograron entrenar un modelo con capacidad de generalización a distintas regiones. Este enfoque tiene la ventaja de ser altamente escalable y de permitir el monitoreo de grandes extensiones marinas sin necesidad de misiones submarinas, aunque enfrenta desafíos relacionados con la resolución de las imágenes satelitales y la disponibilidad de datos de referencia para la validación del modelo.

Aunque la mayoría de investigaciones previas se han centrado en especies distintas, como *Posidonia oceanica*, estos estudios han demostrado el potencial del aprendizaje profundo para la segmentación automática en ecosistemas marinos. Estas referencias respaldan la hipótesis que guía este proyecto, que adapta y aplica un enfoque similar centrado en la monitorización y gestión de *Nanozostera noltei*. No obstante, la investigación sobre *N. noltei* sigue estando en una etapa temprana en comparación con *P. oceanica*, lo que representa no solo un desafío debido a la escasez de estudios específicos, sino también una oportunidad para aplicar y validar metodologías avanzadas que contribuyan a mejorar su monitoreo y conservación.

2.2.- Contexto ecológico y marco normativo

A continuación, se presenta el contexto ecológico de esta planta y marco normativo que la regula, con el objetivo de subrayar la importancia de desarrollar herramientas eficientes para la monitorización de *Nanozostera noltei*. De este modo, se refuerza la relevancia práctica del presente trabajo dentro del marco de la gestión ambiental.

La *Zostera noltii* y *Nanozostera noltei* son consideradas la misma especie, pero el uso de estos nombres depende de diferentes enfoques taxonómicos, es decir, puede encontrarse nombrada de ambas formas dentro de la literatura botánica. Un aspecto a tener en cuenta es que esta especie no debe confundirse con la *Zostera marina*, ya que ésta sí es otra especie distinta, donde la principal diferencia radica en el ancho de sus hojas, como puede verse en la Figura 2.1. Ambas son un tipo planta marina que, además de darse en otras regiones, abunda en el litoral cantábrico y concretamente se puede observar presencia de ambas en la ría de Villaviciosa aunque la especie más abundante y en la que se centra este proyecto es la *Nanozostera noltei*.

Estas plantas marinas presentan una estructura similar a la de las plantas terrestres, con hojas, tallo y raíces con las que se aferran a suelos fangosos y arenosos. Son abundantes en estuarios y rías no contaminadas, donde forman un ecosistema muy valioso. Las plantas marinas se conocen como '*especies ingenieras*' ya que por sí mismas forman un hábitat para otras especies, entre sus hojas conviven peces e invertebrados, que la utilizan como refugio ante depredadores.



Figura 2.1: Aspecto de una pradera formada por *Zostera marina* (hojas anchas) y *Nanozostera noltii* (hojas estrechas). Fotografía Victor M. Vázquez [1].

Además, desde un punto de vista antropocéntrico tienen gran valor pues nos surten de diversos servicios ecosistémicos. Por ejemplo, al engancharse con sus raíces a los fondos arenosos consigue que en caso de temporal las olas rompan antes de llegar a la costa, actuando como barrera natural y evitando que se desprenda el sustrato arenoso del fondo, por lo que consiguen que el agua se enturbie menos. También son muy importantes porque, estas raíces que crecen hacia dentro de la arena, están compuestas de carbono que la propia planta absorbe del aire, es decir, hacen un proceso por el cual entierran el carbono presente en el aire.

Debido a esto, en los últimos años se han desarrollado leyes y directivas tanto a nivel europeo como a nivel nacional que protegen estos ecosistemas. En los años 70 y 80 las plantas marinas habían sufrido una regresión debido a la contaminación con aguas residuales y obras costeras. Gracias a la nueva normativa parece que la regresión se ha detenido incluso algunas zonas están mejorando. En España, la *N. noltei* se encuentra dentro del Listado de Especies Silvestres en Régimen de Protección Especial (*LESPRE*), donde se recogen las especies que son importantes y hay que preservar, creado por el Ministerio de Transición Ecológica y Reto Demográfico (*MITECO*).

Asimismo, la ría de Villaviciosa también esta protegida bajo normativas que regulan los

espacios naturales protegidos. A nivel europeo ha sido catalogada como Lugar de Interés Comunitario (*LIC*) y también como Zona de Especial Protección para las Aves (*ZEPA*), formando parte de la red de espacios protegidos NATURA2000. Además, a nivel autonómico está reconocida como reserva natural parcial y a nivel internacional como sitio *Ramsar*, un convenio que se encarga de proteger los humedales.

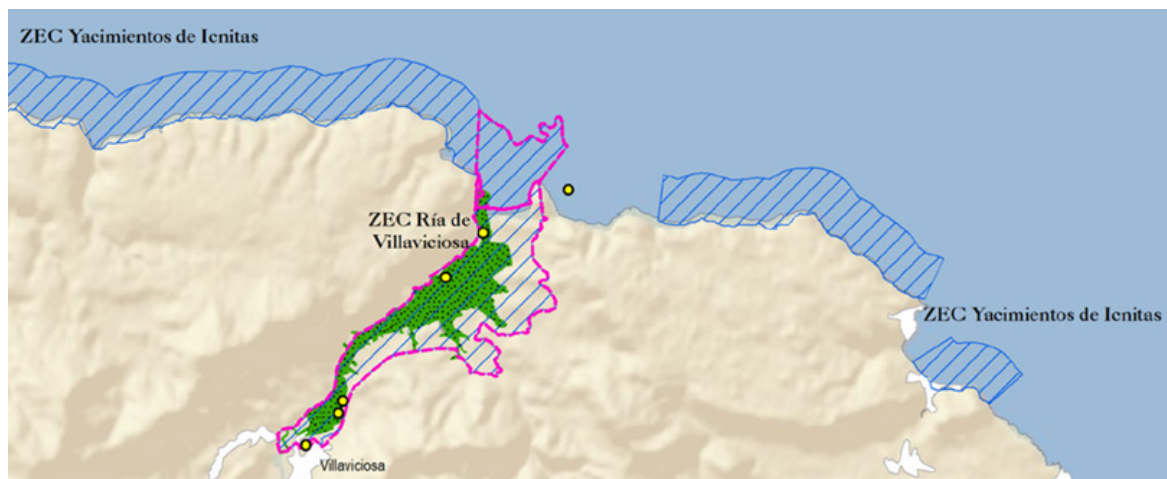


Figura 2.2: Medidas de gestión en la zona de la ría de Villaviciosa. En verde, estuarios con plantas marinas; en rayas diagonales de color azul, zona protegida por *Natura 2000*; en rosa, zona *Ramsar*; puntos amarillo, vertidos urbanos [1].

En este marco, la monitorización de las praderas de *N. noltei* resulta esencial para evaluar la efectividad de las medidas de conservación aplicadas en las zonas protegidas. Los estudios de monitorización persiguen dos objetivos fundamentales: por un lado, determinar la distribución espacial de estas praderas; por otro, analizar su estructura y funcionalidad ecológica.

Mientras que el análisis de la estructura y funcionalidad se basa generalmente en muestreos *in situ*, que permiten obtener datos detallados y muestras para evaluar características específicas de la planta, la identificación y cartografía de su distribución, especialmente en áreas extensas, requiere el uso de imágenes aéreas capturadas con drones. Estos datos se complementan con puntos georreferenciados de presencia y ausencia que facilitan un análisis más exhaustivo y preciso.

Así, el presente estudio se focaliza en la tarea de la detección automatizada de la distribución de *N. noltei*, integrando técnicas de aprendizaje automático con datos aéreos y georreferenciados. De este modo, se sientan las bases para una monitorización sistemática y escalable, capaz de apoyar la gestión efectiva y la conservación de estos ecosistemas marinos.

3. Objetivos concretos y relación con el estado actual

En este capítulo, se presentan en primer lugar los objetivos concretos de la investigación, los cuales buscan evaluar la viabilidad del uso de redes neuronales profundas para la segmentación de praderas marinas y sentar las bases para el desarrollo de métodos más eficientes y escalables. A continuación, se describe el estado actual del proceso de segmentación llevado a cabo por el *IEO*, destacando las limitaciones del enfoque actual en el tratamiento de grandes volúmenes de datos. Finalmente, se expone la solución propuesta, basada en el uso de redes neuronales profundas, remarcando sus ventajas frente al enfoque tradicional.

3.1.- Objetivos concretos

El presente trabajo se concibe como un primer acercamiento exploratorio y una prueba conceptual destinada a evaluar la viabilidad del uso de redes neuronales profundas en la detección y monitorización de las praderas de *N. noltei*. Más que ofrecer una solución definitiva, esta investigación se plantea como un punto de partida sobre el que se podrán construir metodologías más avanzadas que optimicen los procesos actuales y abran nuevas estrategias basadas en inteligencia artificial. La propuesta se centra en el desarrollo de una arquitectura de red neuronal adaptada a la segmentación binaria, la cual permita identificar y delimitar a gran escala las zonas correspondientes a las praderas de *N. noltei*, superando las limitaciones del enfoque manual tradicional. Para lograrlo, se plantea establecer un flujo de trabajo integral que abarque desde el preprocesamiento de las imágenes, etapa clave para garantizar la calidad de los datos de entrada, hasta el entrenamiento y validación del modelo, asegurando así la reproducibilidad de los resultados y la capacidad de adaptación del sistema.

Además, se analizarán diversas configuraciones del conjunto de datos con el fin de determinar el impacto de estos ajustes en la precisión y eficiencia del modelo. Este análisis ayudará tanto a identificar las condiciones óptimas para el entrenamiento, como a proporcionar criterios para la toma de decisiones en la adquisición y procesamiento de datos en futuras aplicaciones, buscando un equilibrio entre el rendimiento del modelo y los costes operativos asociados a la adquisición de las imágenes mediante vuelos con dron.

La experiencia obtenida a partir de este trabajo experimental será determinante para identificar tanto las fortalezas como las limitaciones del enfoque propuesto, así como las áreas de mejora, proporcionando una base sólida para futuras investigaciones. En definitiva, el trabajo aspira a sentar las bases para el desarrollo de herramientas más robustas y eficaces que

permitan una toma de decisiones más informada y un manejo óptimo de las praderas de *N. noltei*, contribuyendo así al avance de la aplicación de la inteligencia artificial en la ecología marina.

3.2.- Estado actual de la segmentación de praderas marinas

En esta sección se explicarán los métodos que actualmente emplean desde el Centro Oceanográfico de Gijón para conocer las regiones de *N. noltei* y los inconvenientes que estos presentan.

Por un lado, los métodos empleados en el muestreo in situ, como se explicó en el capítulo anterior, se basan en la visualización e identificación directa de la pradera recorriendo la ría. Este enfoque tiene la ventaja de ser muy preciso, ya que no hay margen de error en el etiquetado. Sin embargo, su principal desventaja es el tiempo que requiere, especialmente considerando la gran extensión de la ría y el hecho de que solo puede llevarse a cabo durante la marea baja, lo que limita aún más el tiempo disponible. Todo esto hace que poner en práctica este enfoque resulte complicado.

En el caso de las imágenes tomadas por el dron, la metodología de segmentación actualmente empleada por el IEO para identificar las praderas de *N. noltei* se basa, principalmente, en un proceso manual. Un operario se encarga de segmentar a mano las zonas de pradera en las distintas imágenes. Este enfoque presenta importantes limitaciones, como un alto consumo de tiempo, la dependencia del conocimiento y la experiencia del operador, y una mayor probabilidad de errores humanos derivados del cansancio o la falta de motivación. Además, la precisión del trabajo depende en gran medida de la resolución utilizada: aplicar un mayor nivel de zoom mejora la precisión, pero incrementa significativamente el tiempo necesario; por el contrario, trabajar sin hacer zoom reduce el tiempo de segmentación, pero también disminuye la precisión, especialmente en los bordes. Por todo ello, encontrar un equilibrio entre tiempo y calidad es complicado con los métodos actuales.

Además, la segmentación manual no es escalable cuando se trabaja con grandes volúmenes de datos. La única forma de ampliar este proceso sería mediante la contratación de más personal, algo que no siempre es viable debido a las restricciones presupuestarias. Como alternativa, la tarea podría delegarse a otros empleados, lo que implicaría reasignar recursos humanos y apartarlos de sus funciones principales. Esto podría provocar retrasos o una falta de tiempo para abordar otros proyectos, agravando aún más el problema. En consecuencia, el enfoque manual resulta ineficiente para aplicaciones con una elevada cantidad de datos que requieren rapidez y capacidad de escalado.

3.3.- Solución propuesta: Segmentación con redes neuronales profundas

En esta sección se presenta el enfoque propuesto para la monitorización, basado en redes neuronales profundas. Se explicará brevemente qué son las redes neuronales profundas y cómo son capaces de aprender patrones a partir de grandes volúmenes de datos. Además, se realizará una breve revisión de las redes neuronales convolucionales (*CNNs*, por sus siglas en inglés), que serán la arquitectura elegida para este proyecto, debido a su eficacia en tareas de segmentación. A continuación, se detallará la tarea a resolver: la segmentación binaria, y cómo las redes neuronales profundas se aplican eficazmente a esta tarea. Finalmente, se expondrán las ventajas que presenta el uso de esta técnica en comparación con el enfoque actual.

El aprendizaje automático o *machine learning* es un campo de la inteligencia artificial que permite a los sistemas aprender patrones a partir de los datos y hacer predicciones sin ser programados explícitamente para ello.

El aprendizaje profundo o *deep learning* es una subrama del aprendizaje automático, como se ilustra en la Figura 3.1. Los modelos de aprendizaje profundo se basan en el perceptrón, una unidad básica introducida en 1957 por Frank Rosenblatt, para formar redes neuronales profundas (*DNNs*, por sus siglas en inglés). Estas redes consisten en múltiples capas de neuronas artificiales interconectadas, y están diseñadas para aprender representaciones jerárquicas de los datos, lo que les permite identificar patrones complejos y abstractos. A diferencia de los métodos tradicionales de *machine learning*, que dependen en gran medida de la selección manual de características, los métodos de *deep learning* pueden extraer automáticamente características relevantes de los datos sin necesidad de extracción manual de estas, lo que es especialmente útil cuando se trabaja con grandes volúmenes de información no estructurada. Este enfoque ha demostrado ser particularmente eficaz en tareas como la segmentación de imágenes, clasificación y otras aplicaciones que requieren el análisis de grandes cantidades de datos.

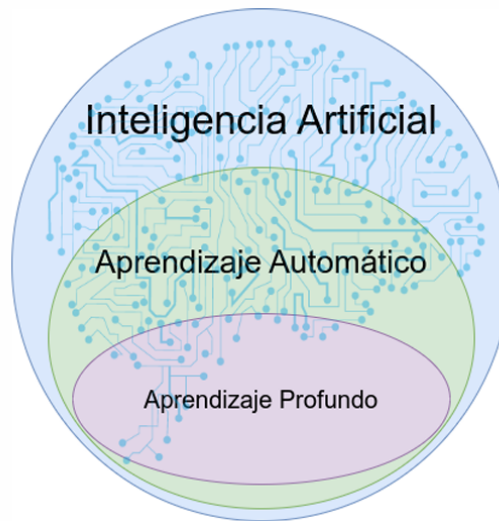


Figura 3.1: Diagrama de los diferentes campos de la inteligencia artificial.

Aunque las redes neuronales tradicionales suelen utilizar capas completamente conectadas, existen otras arquitecturas que emplean capas convolucionales (*CNNs*), que se utilizan ampliamente en tareas de procesamiento de imágenes, como es el caso de este trabajo. En una red completamente conectada, cada neurona de una capa está conectada a todas las neuronas de la capa siguiente (véase Figura 3.2), lo que implica que la salida de cada neurona es una combinación lineal de todas las entradas. Esto lleva a la red a aprender un gran número de parámetros, lo que incrementa el coste computacional. Además, cuando se procesan imágenes, este tipo de capas requieren convertirlas en vectores, lo que provoca una pérdida de la información espacial.

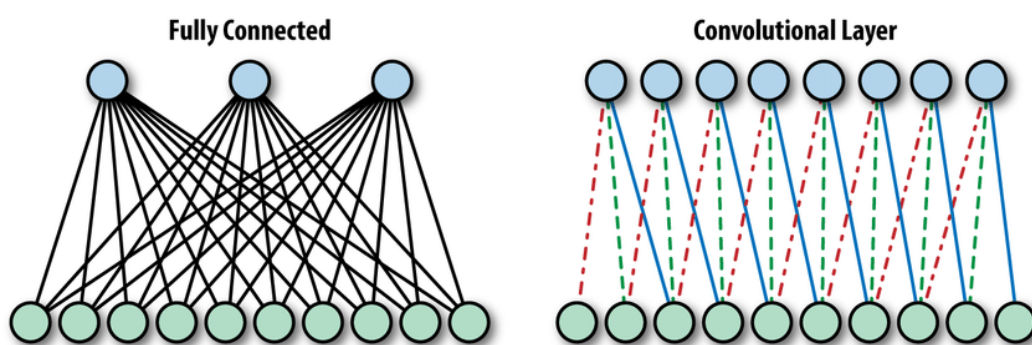


Figura 3.2: Diferencia entre capa totalmente conectada (izquierda) y capa convolucional (derecha) [2].

Por otro lado, las capas convolucionales funcionan de manera diferente: en lugar de conectar cada neurona a todas las entradas, cada neurona en una capa convolucional está conectada solo a un subconjunto de las entradas (véase Figura 3.2), formando un filtro que se

desplaza sobre la imagen (proceso conocido como convolución). Esto reduce el número de parámetros a aprender y, por ende, el coste computacional, lo que permite incluir una mayor cantidad de capas ocultas entre la capa de entrada y la capa de salida. Este mayor número de capas en una red convolucional facilita la captura de patrones locales y espaciales, además de que no es necesario convertir la matriz de la imagen en un vector, lo que preserva la estructura espacial y mejora la capacidad para identificar detalles como bordes, texturas y formas de manera más eficiente. En este proyecto, el enfoque se centrará en el uso de *CNNs* debido a las ventajas que acaban de ser mencionadas y su extendido uso en problemas similares.

El entrenamiento de estas redes se realiza utilizando algoritmos de optimización, como el descenso de gradiente, que ajustan los pesos de las conexiones entre las neuronas con el objetivo de minimizar una función de error, normalmente una función de pérdida que mide la diferencia entre las predicciones de la red y las etiquetas reales de los datos. Así, la red aprende a identificar patrones y relaciones dentro de los datos, lo que le permite ir perfeccionando las predicciones para reducir la penalización por parte de la función de error.

En el caso de la segmentación binaria, la red neuronal profunda recibe una imagen de entrada y, mediante su estructura de capas, produce una imagen de salida donde asigna a cada píxel un valor de pertenencia a una de las dos clases específicas, por ejemplo '*N. noltei*' o '*no N. noltei*', como se puede ver en la Figura 4.7, donde los píxeles con presencia de *N. noltei* se les asigna el valor 1; 0 al resto. Las redes neuronales profundas son particularmente eficaces en tareas de segmentación debido a su capacidad para extraer características locales (como bordes y texturas) a través de las primeras capas y capturar relaciones globales y contextuales a través de las capas más profundas.

Como bien se ha dicho, las capas convolucionales destacan en problemas relacionados con la visión por computador, entre ellos la segmentación. Las primeras redes que sustituyeron las capas totalmente conectadas por capas totalmente convolucionales se conocen como *Fully Convolutional Networks (FCNs)* [3]. Estas redes fueron pioneras en la segmentación de imágenes al permitir procesar imágenes de cualquier tamaño y generar mapas de segmentación de la misma dimensión que la imagen de entrada.

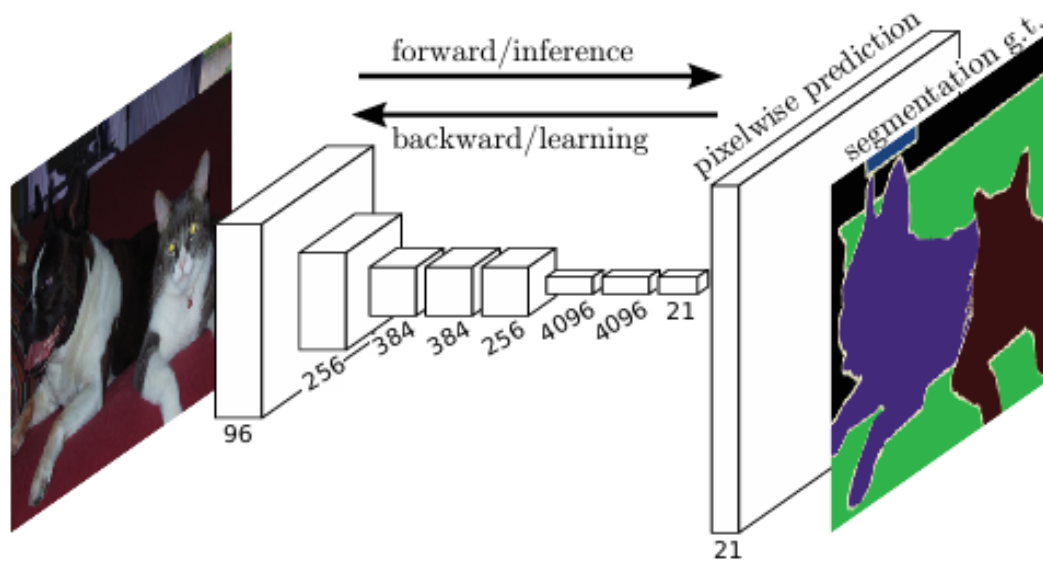


Figura 3.3: Arquitectura de *FCN* presentada en el artículo [3].

Como se observa en la Figura 3.3, la arquitectura de una *FCN* se compone de las siguientes tres partes principales. Aunque fue diseñada para segmentación semántica multiclase, su estructura base es igualmente válida para segmentación binaria, variando únicamente la capa de salida y la función de activación.

- Un *encoder* basado en redes convolucionales, que pueden ser preentrenadas. Se encarga de extraer características progresivamente más abstractas de la imagen.
- Un *decoder*, que usa convoluciones transpuestas para recuperar la resolución original de la imagen y generar el mapa de segmentación.
- Predicción a nivel de píxel, que consiste asignar una etiqueta a cada píxel de la imagen en lugar de producir una única predicción global. Esto significa que la red genera un mapa de segmentación con la misma resolución que la imagen de entrada, donde cada píxel recibe una categoría específica.

Dentro de este tipo de redes, pioneras en el empleo de convoluciones para procesamiento de imagen, se fueron desarrollando otras arquitecturas como *U-Net* [4]. Esta red se caracteriza por su estructura en forma de 'U', que integra dos bloques complementarios. En la parte izquierda de la Figura 3.4 se observa el bloque de *downsampling* o *encoder*, donde se realizan sucesivas operaciones de convolución y *pooling* para extraer características cada vez más abstractas y contextuales. Por otro lado, la parte derecha representa el bloque de *upsampling* o

decoder, que utiliza convoluciones transpuestas para recuperar la resolución espacial perdida. Además, las *skip connections* unen capas de igual nivel, combinando la información de alto nivel del *encoder* con los detalles espaciales del *decoder* mejorando así la precisión en la segmentación.

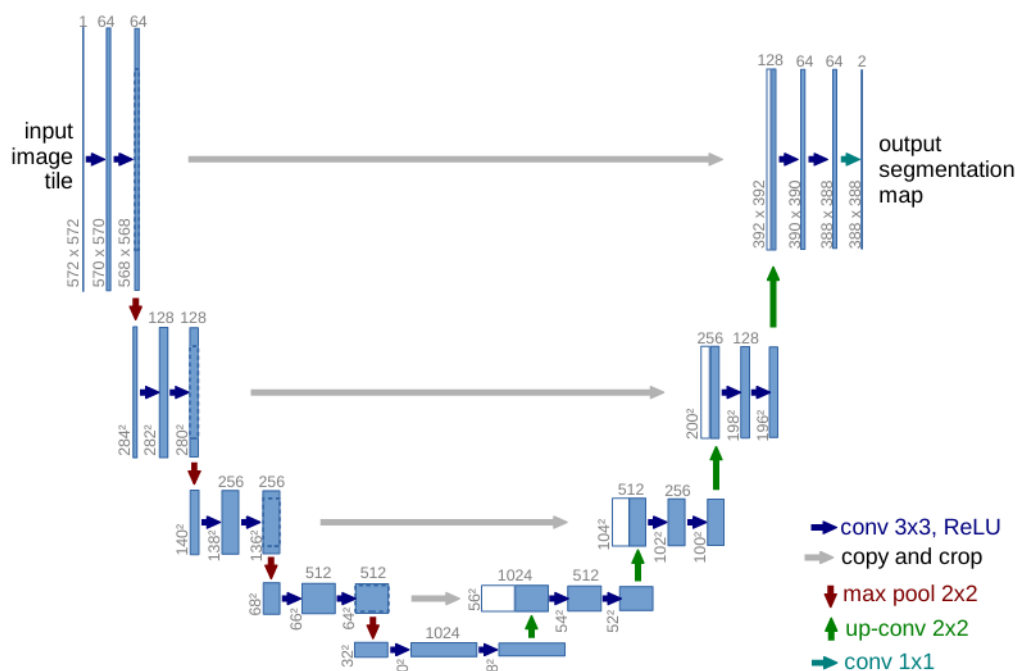


Figura 3.4: Arquitectura *U-Net* [4].

Dentro del bloque de redes convolucionales cabe mencionar también otra con gran nombre dentro del campo de visión por computador como es *DeepLabv3+* [5]. Esta arquitectura es una evolución de la familia *DeepLab* que mejora la segmentación mediante el uso de convoluciones dilatadas que permiten aumentar el campo receptivo sin pérdida de resolución y un mejor mecanismo de decodificación para una mayor precisión en los bordes. Como se observa en la figura 3.5, su arquitectura combina el módulo *Atrous Spatial Pyramid Pooling (ASPP)* que se encarga de capturar información en múltiples escalas al aplicar convoluciones dilatadas con diferentes tasas de dilatación.

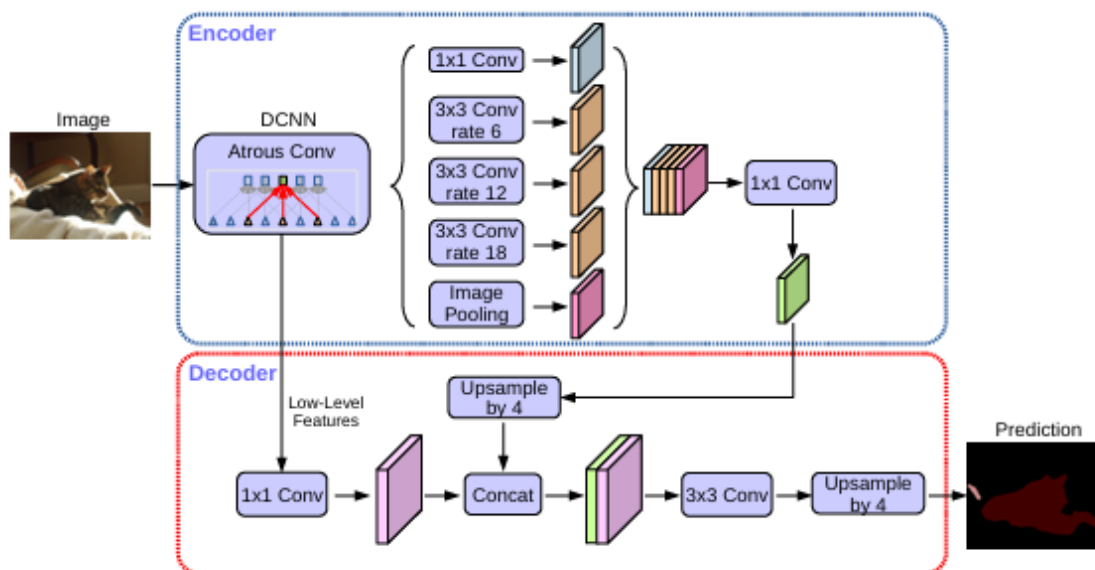


Figura 3.5: Arquitectura DeepLabv3+ [5].

Dado el objetivo de segmentar binariamente praderas de *N. noltei*, se opta por utilizar una red con estructura en 'U' frente a otras alternativas como *FCN* o *DeepLabv3+*. Las *FCN*, aunque pioneras en el uso exclusivo de capas convolucionales para segmentación, pueden presentar dificultades para recuperar con precisión la resolución espacial original, lo que puede comprometer la calidad de los contornos. Por otro lado, *DeepLabv3+* está más orientado a tareas de segmentación semántica con múltiples clases, mientras que *U-Net* fue diseñada específicamente para segmentación binaria en contextos biomédicos [4]. Además, en estudios previos centrados en la segmentación de praderas marinas a partir de imágenes satelitales, este tipo de arquitectura ha demostrado un rendimiento competitivo frente a otras alternativas, posicionándose entre las más eficaces del análisis comparativo realizado en dicho trabajo [9]. Al ser aplicada al caso de las praderas de *N. noltei*, se espera que ofrezca una solución a las limitaciones del método manual tradicional, obteniendo una serie de beneficios significativos:

- **Automatización y eficiencia:** Las redes neuronales profundas tienen la capacidad de procesar grandes volúmenes de datos de imágenes en tiempos considerablemente reducidos, eliminando la necesidad de realizar segmentaciones manuales. Esto no solo acelera el flujo de trabajo, sino que también permite que los recursos humanos se dediquen a tareas más críticas y de mayor valor dentro de los proyectos de investigación. Además, a diferencia del enfoque manual, que puede verse afectado por la motivación o el cansancio de los operadores, las redes neuronales profundas aseguran resultados reproducibles.

- **Escalabilidad:** La capacidad de aplicar este enfoque a conjuntos de datos masivos sin un aumento exponencial en el tiempo o los recursos necesarios representa una ventaja clave. Esto resulta especialmente relevante en proyectos de largo plazo o en estudios que involucran monitoreo a gran escala, donde se generan cientos o miles de imágenes de diferentes ubicaciones.
- **Adaptabilidad:** Uno de los aspectos más destacados de las redes neuronales profundas es su capacidad para generalizar y adaptarse a diferentes dominios y tipos de datos si se requiere. Esto significa que, mediante el entrenamiento adecuado y distintas técnicas de adaptación del dominio, los modelos pueden ajustarse para trabajar con imágenes obtenidas bajo diferentes condiciones, como variaciones en la resolución, el tipo de cámara, o incluso diferencias en la iluminación. Esto convierte a este enfoque en una herramienta versátil que puede evolucionar junto con las necesidades del proyecto o aplicarse en contextos similares, como la segmentación de otras especies marinas.

4. Metodología de trabajo

En este capítulo se detalla la metodología seguida para llevar a cabo la investigación, abarcando desde la descripción de los datos de partida y las herramientas software utilizadas, hasta cada una de las fases necesarias para entrenar y validar un modelo capaz de segmentar praderas de *N. noltei*. Se describe en primer lugar el origen de los datos y las características de las imágenes empleadas, así como los entornos de desarrollo y las principales bibliotecas utilizadas a lo largo del proceso.

A continuación, se explican los distintos pasos del flujo de trabajo: el preprocesamiento de las imágenes, la obtención y tratamiento de las máscaras binarias, la organización del conjunto de datos y las decisiones tomadas. Finalmente, se presentan las etapas de entrenamiento del modelo de segmentación, su validación y la evaluación del rendimiento sobre nuevos casos no vistos. Todo el código fuente desarrollado, junto con las funciones implementadas para cada etapa, se encuentra disponible en el repositorio de GitHub asociado a este trabajo.¹

4.1.- Datos

En esta sección se comentará la procedencia de los datos, así como su tipo y formato. Este último aspecto resulta especialmente relevante, ya que conocer en detalle las características de los datos facilitará su manipulación y tratamiento en las etapas posteriores del trabajo.

Los datos de partida fueron proporcionados por el Instituto Español de Oceanografía a través de un repositorio compartido en la nube. En dicho repositorio se encuentran tres elementos fundamentales: por un lado, las imágenes, y por otro, dos archivos *GeoJSON*, uno que actúa como máscara de referencia y otro que se utilizará a modo de validación.

4.1.1.- Imágenes en formato *TIFF*

Por un lado, se dispone de un total de 15 ortomosaicos que cubren en su totalidad la ría de Villaviciosa. Cada uno de ellos ha sido generado mediante técnicas de fotogrametría, a partir de la combinación de fotografías aéreas sucesivas. La Figura 4.1a muestra uno de estos ortomosaicos a modo de ejemplo. Por motivos de claridad y simplicidad, a lo largo de este trabajo se emplearán los términos ortomosaico e imagen de forma indistinta, dado que un ortomosaico es, en esencia, una imagen digital georreferenciada generada a partir de múltiples imágenes individuales corregidas en escala, orientación y perspectiva.

¹<https://github.com/pelayoelvira/zostera>



(a) Escala 1:2100.



(b) Escala 1:100.

Figura 4.1: Ortomosaico de la ría de Villaviciosa correspondiente a la zona de Bornizal.

Además de los canales habituales *RGB*, algunos de estos ortomosaicos incorporan bandas espectrales adicionales, ya que la cámara del dron utilizado permite, si se configura adecuadamente, la captura de hasta 7 bandas diferentes. Este hecho, sumado a la amplia extensión territorial cubierta, da lugar a ortomosaicos de muy alta resolución, como se detalla en la Tabla 4.1. Visualmente, esto se aprecia en la Figura 4.1, donde se comparan dos ortomosaicos a diferentes escalas: (a) escala 1:2100, que proporciona una vista general, y (b) escala 1:100, con un nivel de zoom considerable. Gracias a la elevada resolución espacial, incluso con este nivel de ampliación, las imágenes mantienen una excelente calidad visual, sin pérdida apreciable de detalle ni aparición de pixelado.

Nombre de la imagen	Dimensiones (px)	Tamaño (GB)
Encienona.tif	33065×34779	2.46
Misiego.tif	29603×39965	2.41
Bornizal.tif	30838×24638	2.06
Bornizal2.tif	27201×41016	3.10
Izq1.tif	58496×95296	7.60
Bornizal3.tif	74098×42637	6.73
Molino1.tif	93457×55440	8.08
Molino2.tif	62170×59153	8.62
Izq2.tif	110827×62643	11.71
Izq3.tif	64385×71061	7.45
Izq4.tif	23498×76344	5.02
Dcha_remate1.tif	35976×29648	2.03
Dcha_remate2.tif	63629×83455	10.96
Dcha_remate3.tif	38025×32504	2.87
Albatros_nanoref.tif	72418×85699	13.49

Tabla 4.1: Características de los ortomosaicos originales.

Estas imágenes, tras el proceso de fotogrametría, estarán en formato ráster, específicamente en Tagged Image File Format (*TIFF*, por sus siglas en inglés). Básicamente, los ráster son mapas de bits compuestos por píxeles que, al combinarse, representan una imagen. Cuantos más píxeles se utilicen para recrear una imagen, mayor será su definición [10].

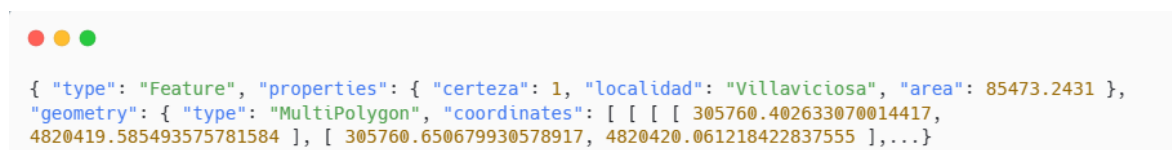
En relación con estas imágenes, el formato *TIFF* resulta especialmente interesante por su uso en imágenes geoespaciales [11]. Entre sus metadatos, se incluye el Sistema de Referencia de Coordenadas, o *CRS* por sus siglas en inglés, un conjunto de parámetros que define cómo se mapean las coordenadas geográficas (latitud y longitud) a coordenadas en una imagen ráster [12]. Es decir, establece la relación entre las coordenadas cartográficas y las coordenadas de los píxeles.

Además, el formato *TIFF* admite compresión sin pérdidas [10], lo que permite conservar la calidad original de la imagen tras ser comprimida y descomprimida, siempre que se utilice este tipo de compresión. Esto resulta especialmente ventajoso para imágenes aéreas de alta resolución, donde la preservación precisa de los detalles es crucial para tareas de visión por computador. Sin embargo, una desventaja es el tamaño de los archivos resultantes, que puede ser considerablemente grande [10], como ocurre en este caso. Otra ventaja destacable es su versatilidad, ya que permite almacenar múltiples bandas, como las *RGB* y otras bandas

espectrales como las mencionadas anteriormente, útiles en distintas aplicaciones.

Por último, el formato *TIFF* es compatible con software de Sistema de Información Geográfica o *SIG* [11], como *QGIS*, ampliamente utilizado para el análisis espacial de imágenes, ya que permite su disposición e interpretación de acuerdo con el espacio terrestre gracias a los diversos parámetros de *CRS* contenidos en los metadatos.

4.1.2.- Máscaras en formato *GeoJSON*



```
{ "type": "Feature", "properties": { "certeza": 1, "localidad": "Villaviciosa", "area": 85473.2431 },  
  "geometry": { "type": "MultiPolygon", "coordinates": [ [ [ [ 305760.402633070014417,  
4820419.585493575781584 ], [ 305760.650679930578917, 4820420.061218422837555 ],... ] ] }
```

Figura 4.2: Segmento del archivo *GeoJSON* *Nanozostera_noltei*.

Por otro lado, entre los datos proporcionados se incluye un archivo *GeoJSON*. Este es un formato de intercambio de datos geospaciales de código abierto que permite representar entidades geográficas simples y sus atributos no espaciales [13]. Basado en JavaScript Object Notation (*JSON*), este formato codifica diversas estructuras de datos geográficos, como puntos, líneas o polígonos, como es el caso. En la Figura 4.2, se puede observar que la estructura de un archivo *GeoJSON* es similar a la de un archivo *JSON*, pero con la adición de información geométrica, como el tipo de elemento (por ejemplo, polígono, línea o punto), junto con las coordenadas geográficas que permiten localizarlo espacialmente.

Uno de los dos archivos de este tipo hace las veces de máscara binaria ya que los polígonos contenidos en este representan las zonas con presencia de la planta de interés. De esta manera, a partir de un único archivo se pueden obtener las máscaras binarias para cada una de las imágenes gracias a que tanto polígonos como ortomosaicos están georreferenciados; este proceso será explicado en la sección 4.3.

Visualmente, la representación que aporta más información no es la estructura del *GeoJSON*, si no el hecho de poder superponer los polígonos sobre las distintas imágenes. Haciendo uso de la herramienta *QGIS*, que se mencionará en la sección 4.2, se puede obtener una representación visual de las imágenes correctamente ubicadas de acuerdo a las coordenadas geográficas, donde se encuentran superpuestas las regiones de interés, permitiendo así ver de manera clara la presencia de *N. noltei* en cada uno de los ortomosaicos. Esto se puede observar en la Figura 4.3, donde el área sombreada en naranja indica la presencia de *N. noltei*.



Figura 4.3: Ortomosaico de una parte de la ría de Villaviciosa con su respectiva zona de interés superpuesta.

Cabe mencionar que el etiquetado de la máscara está hecha a groso modo. Como se explicó en el capítulo 3, uno de los mayores inconvenientes que manejan en el Centro Oceanográfico de Gijón actualmente es que la segmentación manual en imágenes de alta resolución conlleva una gran cantidad de tiempo, sobretodo si se desea hacerla con mucha exactitud. Por lo ello, estas máscaras están realizadas sin detallar precisamente los bordes de *N. noltei*, por lo que cabe la posibilidad de que o bien se esté incluyendo como pradera marina otros elementos o bien se esté dejando zonas de esta fuera de la máscara. Esto será importante tenerlo en cuenta de cara al entrenamiento del modelo, ya que no se buscará una precisión exacta a nivel de píxel. Lo que se buscará es la identificación de regiones de *N. noltei* a gran escala.

Además de este archivo, se dispone de otro *GeoJSON* conocido como *groundtruth*. Este archivo contiene puntos georreferenciados que dentro de sus metadatos indican la presencia o ausencia de *N. noltei* en una región concreta de la ría (véase Figura 4.4). Estas referencias han sido tomadas por investigadores del Centro Oceanográfico de Gijón en algunos de sus muestreos de campo por la ría de Villaviciosa, mediante observación in situ, por lo que son de gran exactitud y precisión.

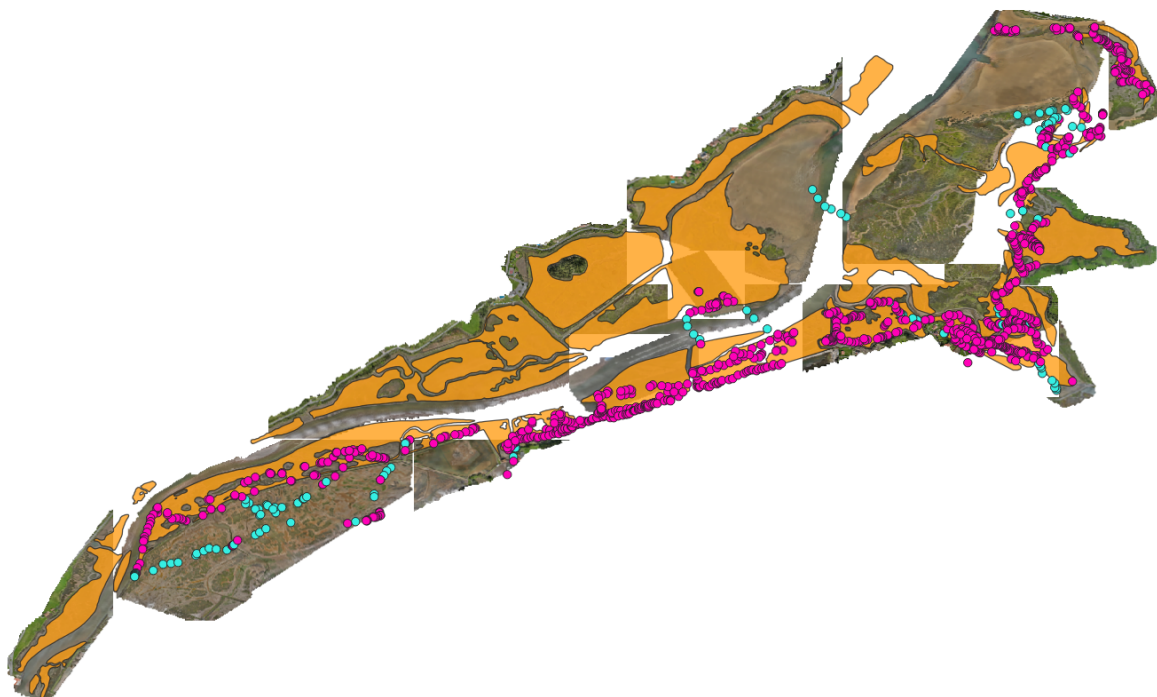


Figura 4.4: Representación del mosaico de la ría de Villaviciosa; superpuesto, en color naranja las regiones de interés; en color rosa los puntos de presencia y en azul los puntos de ausencia.

La distribución de los datos contenidos en el archivo *groundtruth* se resume a continuación. Se cuenta con un total de 1105 puntos georreferenciados, de los cuales 987 indican la presencia de *N. noltei* (positivos) y 118 indican su ausencia (negativos). Como se aprecia visualmente en la Figura 4.5, existe un marcado desbalance de clases. Este desequilibrio, que refleja la predominancia de praderas marinas en la región, puede influir a la hora de evaluar las máscaras predichas, ya que la baja representación de la clase negativa podría conducir a una evaluación sesgada. No obstante, dado que estos puntos se utilizan únicamente para evaluar la calidad de las máscaras y no para el entrenamiento del modelo no es un problema tan extremadamente grave. El mayor inconveniente quizá sea que no existe una distribución homogénea de puntos en cada región de la ría. Como se ve en la Figura 4.4, existen regiones con una gran cantidad de puntos de validación mientras que otras no presentan ninguno y por lo tanto no podrán ser evaluadas de esta forma.

Aún conociendo estas circunstancias, este archivo será de gran utilidad ya que proporcionará un *feedback* más próximo a la realidad de la calidad de las predicciones del modelo. En contraposición con las máscaras, que han sido generadas de manera aproximada y pueden contener errores en la delimitación de las praderas de *N. noltei*, los puntos del *groundtruth* han sido obtenidos mediante observación directa, lo que les confiere una mayor precisión y fiabilidad para la evaluación del desempeño del modelo.

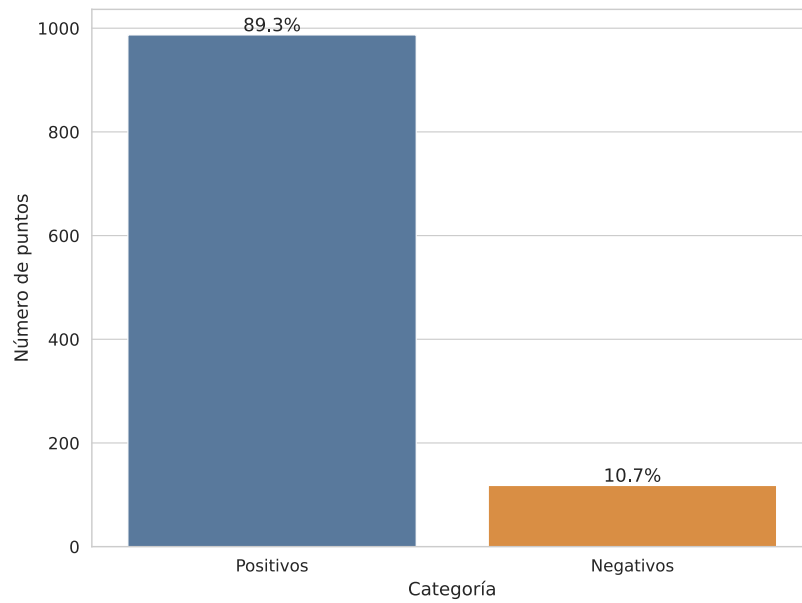


Figura 4.5: Distribución de las clases presentes en el archivo de *groundtruth*

4.2.- Herramientas software

En esta sección se mencionarán algunas de las herramientas software más relevantes para desarrollar el proyecto, bien porque son herramientas menos conocidas y que merece la pena destacar por la importancia para el desarrollo o bien otras herramientas, que aunque están a la orden del día y sean más que conocidas es de recibo mencionarlas.

4.2.1.- Sistema de información geográfica

Como visor de imágenes se empleó **QGIS** [14], un software de código abierto para el análisis y la visualización de datos geoespaciales. Es un sistema de información geográfica (*SIG*) que permite trabajar con datos vectoriales y ráster, ofreciendo una gran variedad de herramientas para gestionar, analizar y representar información geográfica. En este problema, esta herramienta fue útil para poder hacer inspecciones visuales del problema. Gracias a ella fue posible formar un mosaico (véase Figura 4.4) con todos los ortomosaicos, cada uno ubicado en su correcta posición gracias a las coordenadas contenidas en los metadatos. Además, como permite trabajar con capas, fue posible sobreponer los polígonos contenidos en el archivo *GeoJSON* para poder visualizar de forma rápida en cada ortomosaico aquellas zonas con presencia de *N. noltei*.

4.2.2.- Lenguaje de programación y bibliotecas

El lenguaje de programación empleado fue **Python**, con versión 3.9.20. Además se destaca el uso de las siguientes bibliotecas por su importancia para la resolución del problema:

Rasterio [15] es una biblioteca de Python diseñada específicamente para trabajar con datos ráster geoespaciales. Esta herramienta es especialmente útil para leer, escribir y manipular datos ráster junto con sus metadatos geoespaciales.

En este caso, se ha utilizado para abrir las imágenes ráster y acceder a los metadatos asociados. Además, se utilizó para verificar y ajustar el sistema de coordenadas espaciales asegurando la correcta alineación geográfica de las imágenes ráster con las máscaras. Otra biblioteca empleada para el manejo de imágenes ráster es **TiffFile**.

Shapely [16] es una biblioteca de Python para el manejo y análisis de datos geométricos en dos dimensiones utilizada para trabajar con objetos como puntos, líneas, polígonos y multipolígonos. En el contexto de este problema, resulta particularmente útil para definir y manipular con precisión las regiones de interés, ya que en el archivo *GeoJSON* que contiene las máscaras, estas están especificadas como geometrías de tipo *Polygon* o *Multipolygon*.

Además, Shapely proporciona herramientas para realizar operaciones espaciales, como

encontrar intersecciones o uniones entre distintas geometrías. Estas funcionalidades ayudan a obtener las regiones definidas y también a realizar las transformaciones necesarias para procesar los datos geoespaciales.

Como su nombre indica, **GeoPandas** [17] utiliza como base la estructura principal de Pandas, los *DataFrames*, con modificaciones que permiten manejar datos geoespaciales. En este problema, GeoPandas se utiliza para leer y almacenar la información contenida en los archivos *GeoJSON*, ya que facilita el manejo de las geometrías al soportar operaciones, como intersecciones o uniones espaciales entre polígonos con la biblioteca *Shapely* anteriormente comentada.

Dado que las imágenes tienen una alta resolución, no es viable ejecutar ningún modelo directamente sobre ellas. Por esta razón, se recurrió al uso de la biblioteca **Patchify** [18], diseñada para dividir imágenes en segmentos más pequeños, conocidos como *patches*. Un *patch* es una subimagen de tamaño fijo que representa una porción local del contenido original.

4.2.3.- Frameworks

Como frameworks de desarrollo de modelos de *deep learning* se han empleado **Keras** [19] y **TensorFlow** [20]. Ambas bibliotecas han sido fundamentales siendo la columna vertebral ya que se complementan entre sí proporcionando todo lo necesario para la construcción del modelo, como una interfaz intuitiva para el diseño de la red neuronal, herramientas para entrenar, optimizar y evaluar el modelo, soporte para procesamiento en GPU entre otras funcionalidades.

Para el ajuste de hiperparámetros (véase Anexo A) se empleó un framework especializado en este campo como es **Optuna** [21]. Esta biblioteca proporciona diferentes estrategias, como métodos bayesianos o algoritmos evolutivos para optimización de hiperparámetros, que se adaptan dinámicamente a los resultados obtenidos para guiar la exploración del espacio de búsqueda. Optuna realiza la optimización de manera más eficiente que otros métodos gracias a su capacidad de *pruning*, que interrumpe evaluaciones poco prometedoras en las primeras etapas del entrenamiento, reduciendo el tiempo de búsqueda y el uso de recursos computacionales.

Dentro de este framework se encuentra Optuna Dashboard, una herramienta visual integrada que facilita el análisis de los ensayos realizados. Este panel interactivo permite explorar la evolución de la optimización, visualizar la distribución de los hiperparámetros y analizar la relación entre distintas configuraciones y el desempeño obtenido.

4.3.- Preprocesamiento de datos

En esta sección se explicará paso a paso el proceso seguido para el tratamiento de los ortomosaicos. El preprocesado requerido en este trabajo ha sido notable, ya que no ha sido un proceso trivial, sino que ha requerido una atención cuidadosa y detallada. La obtención de las máscaras binarias en formato ráster y la aplicación de ajustes específicos a las imágenes han sido fundamentales para garantizar la calidad de los datos. Además, el ingente tamaño de las imágenes ha añadido complejidad, haciendo necesaria una reducción de la resolución para poder trabajar de manera eficiente. Es por ello que se dedica una explicación detallada a esta parte del proceso, ya que el preprocesamiento ha sido una de las etapas más críticas del proyecto. Un preprocesamiento adecuado es fundamental para garantizar que el modelo reciba datos de alta calidad y correctamente alineados, lo que influye directamente en el rendimiento y precisión. Sin un preprocesado correcto, la segmentación podría verse afectada por datos ruidosos o mal alineados, lo que comprometería los resultados.

4.3.1.- Paso de *GeoJSON* a máscaras binarias en formato ráster

Como se ha comentado en la sección previa, cada ortomosaico no viene etiquetado con su correspondiente máscara binaria en formato ráster, sino que se proporciona un archivo *GeoJSON* con las coordenadas geoespaciales de las zonas de interés dentro los 15 ortomosaicos. Dado que la tarea a resolver es segmentación binaria, es crucial disponer de estas máscaras en formato ráster, luego ese será el primer paso. A continuación, se explica el proceso seguido para conseguir esto, en el cual se ha utilizado como referencia tanto la información como el código contenido en el artículo [12]. También se consultó la fuente [22] aunque esta última está basada en la primera fuente que se ha citado en este párrafo, pues la menciona dentro de su artículo.

Inicialmente, se carga el archivo *GeoJSON* que contiene las máscaras, almacenando su información en un *GeoDataFrame*, el cual actuará como referencia para dicho contenido a lo largo de esta explicación. A continuación, se aplica un filtro para seleccionar únicamente los registros correspondientes a la localidad de Villaviciosa. Esta operación permite centrarse exclusivamente en las geometrías relevantes para este trabajo, ya que el archivo original incluye polígonos de otras zonas del litoral asturiano que no forman parte del área de estudio.

El procedimiento general que se describirá a partir de este punto para generar las máscaras binarizadas se resume de forma estructurada en el Algoritmo 1.

Algorithm 1: Generación de máscaras ráster.

Input: *GeoDataFrame* G filtrado, lista de rásteres $\{R_i\}$, func. auxiliar `poly_from_utm`

Output: Máscaras binarias en formato ráster

```

1 foreach ráster  $R$  en  $\{R_i\}$  do
2    $C \leftarrow \text{CRS}(R)$ ;
3   if  $G.crs \neq C$  then
4      $G \leftarrow \text{reproyectar}(G, C)$ ;
5    $poly\_shp \leftarrow []$ ;
6   foreach registro  $g$  en  $G$  do
7     if  $g.geometry.type \in \{Polygon, MultiPolygon\}$  then
8        $P \leftarrow \text{poly\_from\_utm}(g.geometry, R.transform)$ ;
9        $\text{anexar}(poly\_shp, P)$ ;
10   $mask \leftarrow \text{rasterize}(poly\_shp, \text{out\_shape} = R.shape, \text{fill} = 0, \text{value} = 1)$ ;
11   $\text{escribir\_ráster}(mask \times 255, \text{meta} = R.meta)$ ;

```

El núcleo del proceso inicia iterando sobre todos los ortomosaicos disponibles. Cada archivo ráster se abre y se accede a sus metadatos asociados, entre los que se encuentra el sistema de referencia de coordenadas (*CRS*). Este dato es fundamental, ya que posteriormente se comprueba si el *CRS* del *GeoDataFrame* coincide con el del ráster. Esta coincidencia es imprescindible para garantizar la precisión espacial de las máscaras generadas. En caso de detectar discrepancias entre ambos sistemas, se reproyectan las geometrías del *GeoDataFrame* al *CRS* del archivo ráster.

A continuación, se inicia el proceso de creación de la máscara binaria. Para ello, se declara una lista vacía destinada a almacenar las geometrías correspondientes a las zonas de interés. Mediante un bucle, se itera sobre cada fila del *GeoDataFrame*, donde cada una representa una región con presencia de *N. noltei*. Para cada una de estas regiones, se comprueba si la geometría es de tipo *Polygon* o *MultiPolygon*, ya que ambos son los formatos válidos para la rasterización, pero requieren un tratamiento distinto. Mientras que un *Polygon* representa una única área continua, un *MultiPolygon* agrupa varias áreas independientes bajo una misma entidad.

A continuación, se procede a transformar las coordenadas de las regiones de interés del espacio geográfico al espacio de píxeles del ráster. Aunque previamente ya se ha verificado que el sistema de referencia espacial (*CRS*) del archivo *GeoJSON* coincide con el del ráster (reproyectándolo si fuera necesario), en este punto lo que se reproyecta no es el sistema de coordenadas, sino las propias geometrías.

Algorithm 2: Función auxiliar `poly_from_utm`

Input: Geometría *geom* de tipo *Polygon*/*MultiPolygon*, transformación afín *T*

Output: Lista de polígonos en coordenadas de píxel

```

1 if geom.type = Polygon then
2   E ← []; // puntos del contorno exterior
3   foreach coord en geom.exterior.coords do
4     anexar(E,  $T^{-1} \times \text{coord}$ );
5   H ← []; // lista de huecos internos
6   foreach interior en geom.interiors do
7     h ← [];
8     foreach coord en interior.coords do
9       anexar(h,  $T^{-1} \times \text{coord}$ );
10    anexar(H, h);
11  return [Polygon(E, holes = H)];
12 else
13   L ← [];
14   foreach subpoly en geom.geoms do
15     anexar(L, poly_from_utm(subpoly, T));
16  return L;

```

Para ello, se emplea una función auxiliar (véase Algoritmo 2) que toma como entrada una geometría (de tipo *Polygon* o *MultiPolygon*) y la transformación afín del ráster. Esta transformación, obtenida de los metadatos, permite mapear las coordenadas cartográficas de las geometrías al sistema de coordenadas del ráster, es decir, al espacio de píxeles de la imagen. Para ello, se aplica su inversa a cada vértice de la geometría, ya que por defecto la matriz está orientada a convertir de píxeles a coordenadas geográficas. De este modo, se garantiza la correcta alineación entre las geometrías vectoriales del *GeoDataFrame* y la cuadrícula del archivo ráster.

Dependiendo del tipo de geometría, la función trata de forma diferente el contorno exterior y los huecos internos: si se trata de un *Polygon*, se transforma su contorno y cada hueco interno; si se trata de un *MultiPolygon*, se aplica recursivamente el mismo proceso a cada uno de los polígonos que lo componen. Las geometrías transformadas resultantes, ya expresadas en coordenadas de píxel, se añaden a la lista que se utilizará posteriormente en el proceso de rasterización.

A partir de aquí se convierten las formas poligonales en una matriz ráster, donde cada celda de la matriz puede contener un valor que indica si el píxel será o no *N. noltei*, es decir,

será binaria, indicando con un valor de 1 las zonas de interés. Además, esta matriz es del mismo tamaño que la imagen original. Tras este proceso ya se tendría una máscara binaria en formato ráster, como se puede ver en la Figura 4.7. La Figura 4.6 representa la sección de la ría respectiva a la máscara que aparece a su derecha. Como se puede ver, la máscara generada es idéntica ya que refleja las zonas de interés de manera precisa, manteniendo la misma forma y ubicación de las geometrías representadas en el archivo vectorial.



Figura 4.6: Imagen de un sector de la ría de Villaviciosa; superpuesta la máscara en formato *GeoJSON*



Figura 4.7: Máscara asociada a la imagen de la ría 4.6

4.3.2.- Reducción de la resolución

Debido a la necesidad de trabajar con datos rasterizados de alta resolución, se implementó un proceso de remuestreo para ajustar la resolución espacial de los datos a un valor deseado. En este caso, se definió una resolución que depende de un parámetro que indica la nueva resolución expresada en grados ya que el sistema de coordenadas es *WGS 84*, que utiliza grados de latitud y longitud como unidades. Inicialmente se estableció un valor de 0.0000006° , lo que equivale a que cada píxel mide 0.05 metros.

Para determinar las dimensiones del ráster con la nueva resolución, primero se obtiene la resolución actual de los metadatos (en grados, para coordenadas geográficas). Con esta información, se obtiene un factor de escala dividiendo la resolución actual por la resolución deseada. Este factor se utiliza para calcular el nuevo ancho y alto del ráster en píxeles, asegurando que la transformación se ajuste proporcionalmente a su tamaño original.

El remuestreo de los datos se realiza utilizando una interpolación bilineal, que interpola los valores de los píxeles para suavizar los cambios en la resolución. Los datos ya remuestreados, finalmente, se escriben en un nuevo archivo ráster, especificando el nuevo tamaño, la transformación ajustada y el sistema de coordenadas original.

El mismo proceso se aplica para las máscaras binarias con la diferencia de que se aplica una interpolación por el vecino más próximo debido a que el propósito de las máscaras binarias es preservar únicamente dos valores discretos (1 y 0). La interpolación por vecino más próximo asegura que cada píxel de la máscara remuestreada adopte el valor del píxel más cercano del ráster original para evitar la generación de valores intermedios que no tendrían sentido en datos binarios.

Tras esto, dado que solo cuatro ortomosaicos cuentan con 7 bandas, se optó por seleccionar únicamente las tres primeras bandas (*RGB*) y eliminar las restantes. Esta decisión se tomó para garantizar que todas las imágenes tengan tres canales, lo que evita problemas futuros derivados de la diferencia en el número de bandas entre las imágenes.

Nombre de la imagen	Dimensiones (px)	Tamaño (MB)
Encienona.tif	8448×8886	124.59
Misiego.tif	7583×10237	123.08
Bornizal.tif	8471×6767	118.98
Bornizal2.tif	10947×16507	389.51
Izq1.tif	12962×21116	298.20
Molino1.tif	21772×12915	340.37
Molino2.tif	20583×19584	725.84
Izq2.tif	36753×20774	992.19
Izq3.tif	21853×24119	658.69
Izq4.tif	5494×17851	217.67
Dcha_remate1.tif	8937×7365	98.13
Dcha_remate2.tif	18269×23962	694.35
Dcha_remate3.tif	10766×9203	179.46
Albatros_nanoref.tif	5282×6251	53.47

Tabla 4.2: Características de los ortomosaicos tras aplicar el proceso de reducción de resolución.

Con estas modificaciones y ajustes, se han obtenido ortomosaicos con una resolución reducida y consistentes en cuanto a la cantidad de bandas. La resolución final seleccionada (0.05 metros por píxel) representa un equilibrio adecuado entre nivel de detalle espacial y

viabilidad computacional, permitiendo mantener la información relevante para la detección de praderas sin comprometer la eficiencia en etapas posteriores. Las características finales de los ortomosaicos, incluyendo sus dimensiones y tamaños, se resumen en la Tabla 4.2.

4.3.3.- Posprocesado de máscaras

Una vez completado el preprocesamiento descrito anteriormente, fue necesario aplicar un posprocesado adicional a algunas máscaras binarias. Esto se debe a la propia naturaleza de los ortomosaicos, los cuales, como se explicó anteriormente, son el resultado de unir múltiples imágenes captadas por el dron. Como consecuencia, la representación real de la ría suele tener una forma irregular, mientras que el ráster final adquiere una geometría rectangular o cuadrada al añadirse automáticamente un relleno blanco para completar las dimensiones necesarias, como se aprecia en la Figura 4.8.

Este relleno puede provocar que zonas vacías, que no contienen información útil, sean confundidas con regiones de interés en las máscaras correspondientes. Además, dado que los ortomosaicos representan zonas colindantes, puede haber solapamientos entre ellos. Esto puede generar situaciones en las que zonas etiquetadas como interés dentro de una máscara realmente pertenezcan a otro ortomosaico adyacente. Un ejemplo de esto se puede observar en la Figura 4.9, donde aparecen regiones segmentadas en la esquina superior derecha que no se corresponden con ninguna estructura visible en la imagen original de la Figura 4.8, sino con el área blanca de relleno.

Para corregir esta situación, se construyó una máscara booleana que identifica los píxeles correspondientes al área de relleno (es decir, aquellos que tienen valor de intensidad máximo en la imagen RGB original). A partir de esta máscara, se filtraron las regiones mal etiquetadas en la máscara binaria, asignando a estos píxeles el valor 0 (negro), de modo que quedaran excluidos como zonas de interés. El resultado de este filtrado se muestra en la Figura 4.10, donde dichas regiones han sido correctamente eliminadas.

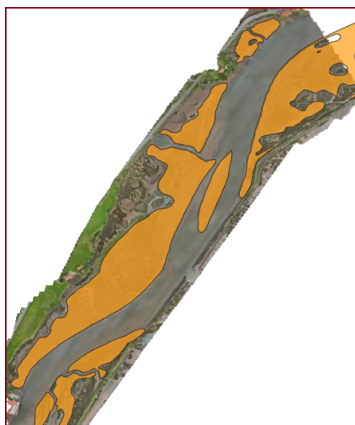


Figura 4.8: Imagen de un sector de la ría de Villaviciosa.



Figura 4.9: Máscara sin procesar asociada a la Figura 4.8



Figura 4.10: Máscara procesada asociada a la imagen de la Figura 4.8

4.3.4.- Obtención y filtrado de *patches*

Aunque las imágenes ya han sido redimensionadas, siguen siendo demasiado grandes para ser utilizadas directamente como entrada del modelo. Por ello, se optó por dividir tanto las imágenes como sus máscaras correspondientes en fragmentos más pequeños o *patches*, mediante ventanas de tamaño fijo sin solapamiento. Para facilitar esta división sin necesidad de aplicar *padding*, las imágenes y máscaras fueron recortadas previamente de forma que sus dimensiones fueran múltiplos exactos del tamaño de los *patches*. Esta decisión se justifica por el hecho de que los bordes de los ortomosaicos suelen carecer de información relevante, como se observa en la Figura 4.8. Como resultado, de cada imagen se obtuvieron múltiples *patches* únicos de tamaño 512×512 píxeles. El número exacto de *patches* generados varía en función de las dimensiones de cada imagen, y cada uno fue almacenado junto con su máscara correspondiente e identificado de forma individual, permitiendo conservar la trazabilidad respecto a la imagen original.

No obstante, debido a la naturaleza de estos ortomosaicos, muchas de estas imágenes no contienen información útil en toda su superficie. Esto es particularmente evidente en las zonas periféricas, donde frecuentemente no aparece la ría, como se muestra en la Figura 4.8. Por tanto, incluir *patches* vacíos en el entrenamiento podría introducir ruido o sesgos, al no aportar información relevante para la detección de praderas de *N. noltei*. Con el objetivo de preservar únicamente aquellos *patches* con contenido significativo, se aplicó un proceso de filtrado. Este consistió en normalizar cada *patch* y calcular el promedio de sus valores de intensidad para identificar aquellos que tenían valor de intensidad máximo, siendo estos eliminados junto con su correspondiente *patch* de máscara. Finalmente, tras este proceso, se obtuvieron un total de 6396 *patches* válidos, con dimensiones $512 \times 512 \times 3$ para las imágenes

y 512×512 para las máscaras extraídos de 14 de los 15 ortomosaicos disponibles. Uno de ellos se reservó para ser utilizado exclusivamente en tareas de inferencia, funcionando como caso no visto en futuras pruebas.



(a) Patch de imagen original.



(b) Patch de la máscara correspondiente.

4.4.- Estructura y preparación del flujo de entrenamiento

Una vez completado todo el preprocesamiento descrito en las secciones anteriores, los datos están listos para ser utilizados. En esta sección se detalla el diseño del módulo encargado de la carga de los *patches*, así como la configuración general del *pipeline* de entrenamiento. Es decir, se aborda la infraestructura necesaria para entrenar el modelo de manera eficiente, excluyendo por el momento tanto la arquitectura del modelo como el propio entrenamiento, los cuales se tratarán en detalle en el capítulo siguiente.

4.4.1.- Carga de los datos

El proceso de carga y preparación de los datos comienza con la identificación y organización de los archivos de imágenes y sus correspondientes máscaras. Para asegurar que cada imagen se asocie correctamente con su máscara, se ordenan ambos conjuntos de archivos alfabéticamente para evitar desajustes que podrían surgir si el orden de las imágenes y las máscaras no coincide en los directorios. A continuación, se leen las imágenes y se convierten a un formato de mayor precisión como *float32*, ya que tras todo el preprocesado los *patches* estaban en *int8*. Además de esto se les aplica una normalización, lo que facilita el entrenamiento del modelo, ya que trabajar con valores escalados de manera uniforme mejora la convergencia del modelo. Este mismo proceso se aplica sobre las máscaras. En caso de que una máscara sea bidimensional, es decir, tenga forma (H, W) , se le añade una dimensión adicional para que adopte la forma $(H, W, 1)$. Esta nueva dimensión no representa un canal adicional en el sentido habitual de las imágenes RGB, sino que permite mantener una misma estructura entre imágenes y máscaras.

Toda esta carga y normalización se realiza en paralelo utilizando métodos de *TensorFlow*, lo que optimiza el flujo de datos y mejora la eficiencia del proceso de preprocesamiento durante el entrenamiento. Al cargar las imágenes y máscaras de manera dinámica y paralela, se aprovechan al máximo las capacidades de procesamiento, reduciendo la carga computacional y evitando posibles problemas de memoria.

Una vez completado este proceso se tendrán tres subconjuntos: entrenamiento, validación y prueba. La partición se realiza de manera aleatoria, pero utilizando una semilla que garantiza la reproducibilidad. Las proporciones de los subconjuntos son predefinidas: 70 % para entrenamiento, 20 % para validación y 10 % para prueba. Estos subconjuntos se organizan como *Datasets* de *TensorFlow*, donde los datos ya se han convertido a tensores, listos para ser utilizados en el entrenamiento del modelo. En la Tabla 4.3 se muestran las particiones y la cantidad de datos por cada subconjunto.

Conjunto	Porcentaje	Cantidad de <i>patches</i>
Entrenamiento	70 %	4476
Validación	20 %	1280
Prueba	10 %	640
Total	100 %	6396

Tabla 4.3: Distribución de imágenes por conjunto

4.4.2.- Flujo de entrenamiento

En esta subsección se detallará el *pipeline* completo de entrenamiento, comenzando desde la preparación eficiente de los *datasets* hasta la configuración final seleccionada para el entrenamiento del modelo.

Una vez que los datos están estructurados en *datasets*, se procede a dividirlos en lotes (*batches*). Esta división permite procesar las imágenes en pequeñas cantidades durante el entrenamiento, lo que mejora la eficiencia computacional al reducir la memoria necesaria en cada iteración. De este modo, se pueden procesar grandes volúmenes de datos sin sobrecargar la memoria de la GPU. Además, permite realizar actualizaciones más frecuentes de los pesos, acelerando la convergencia del modelo. Junto con la división en lotes, se emplea una técnica llamada *prefetch* para optimizar la carga de datos durante el entrenamiento. Esta técnica permite que la carga de datos ocurra en paralelo al procesamiento del modelo, evitando que el entrenamiento se detenga mientras se preparan los datos. Al tener los datos listos con antelación, se reduce el tiempo de inactividad del modelo, lo que acelera el flujo de entrenamiento.

El siguiente paso es preparar el entrenamiento del modelo. A continuación, se explicarán los aspectos relacionados con la configuración del entrenamiento. Es importante destacar que, aunque la arquitectura elegida para los experimentos se tratará en el capítulo 5, las decisiones sobre la configuración del entrenamiento se han tomado teniendo en cuenta dicha arquitectura.

4.4.2.1.- *Callbacks*

El primer aspecto a considerar son los *callbacks*, métodos que se ejecutan automáticamente en puntos específicos durante el entrenamiento del modelo y que son muy útiles para tareas como la monitorización del rendimiento. En este caso, se han empleado los siguientes *callbacks*:

- **EarlyStopping**: Este método se utiliza para detener el entrenamiento cuando el modelo deja de mejorar después de un número determinado de épocas. En particular, si durante 30 épocas consecutivas no se observa una mejora mínima de 0.000001 en el error de validación, el entrenamiento se detendrá automáticamente. Esto ayuda a prevenir el sobreajuste y a ahorrar tiempo de entrenamiento innecesario.
- **ReduceLROnPlateau**: Este método al igual que el mencionado previamente también monitorea el error en validación. En concreto, cuando este deja de mejorar tras 10 épocas, se reduce la tasa de aprendizaje en un 0.1. Esto ayuda a evitar que el modelo se estanque y le permite escapar de posibles mínimos locales, facilitando su convergencia.
- **TensorBoard**: Este método permite guardar información sobre el error y el resto de métricas de rendimiento empleadas durante el entrenamiento en archivos que luego pueden ser analizados a través de *dashboards* interactivos.
- **ModelCheckpoint**: Este método permite almacenar automáticamente los pesos del modelo durante el entrenamiento, asegurando que se guarden las mejores versiones según el error en validación. Esto facilita la recuperación del mejor modelo sin necesidad de entrenarlo nuevamente y permite continuar el entrenamiento desde un punto intermedio si fuese necesario.
- **SavePredictionsCallback**: Se diseñó este método personalizado con el objetivo de obtener más información durante el proceso de entrenamiento. Este se encarga de almacenar los 5 primeros *patches* de cada época, junto con sus respectivas máscaras reales y predichas. De este modo, se puede seguir el proceso de aprendizaje de manera más visual y observar cómo las predicciones del modelo mejoran progresivamente. La razón de crear este *callback* fue que, al principio, el modelo solo predecía prácticamente fondo, lo cual denotaba que había algún problema. Gracias a este seguimiento visual, fue más sencillo identificar el problema y verificar que el modelo estaba aprendiendo correctamente, mejorando sus predicciones con el tiempo.
- **TimeLoggingCallback**: Por último, este método se diseñó para registrar y mostrar el tiempo que tarda cada época y el tiempo total de entrenamiento. Esto se hizo con el objetivo de monitorear el rendimiento temporal del entrenamiento y facilitar comparaciones entre distintas configuraciones del modelo y de los datos empleados.

4.4.2.2.- Selección del optimizador

A la hora de compilar el modelo, se utiliza como función de optimización *AdamW* [23], una variante del ampliamente conocido *Adam*. *AdamW* fue diseñado específicamente para abordar problemas relacionados con la regularización en el entrenamiento de modelos. En

Adam, la regularización $L2$ se añade directamente a la función de pérdida, penalizando los pesos grandes al agregar un término proporcional al cuadrado de la norma de los pesos. Esta penalización influye en el cálculo del gradiente durante la optimización.

Por otro lado, *AdamW* aplica un enfoque diferente al desacoplar la regularización del proceso de actualización del gradiente. En lugar de incluir el término de regularización en la función de pérdida, el *weight decay* en *AdamW* se aplica directamente sobre los parámetros del modelo después de cada actualización, lo que penaliza el crecimiento excesivo de los pesos de manera más eficiente. Este enfoque mejora la estabilidad del entrenamiento, facilita la convergencia y ofrece un mejor equilibrio entre la optimización y la regularización, reduciendo así el riesgo de sobreajuste. Por estas razones y porque demostró un buen desempeño en pruebas iniciales, se eligió *AdamW* en lugar de la versión estándar de *Adam*, a pesar de que esta última es más comúnmente utilizada.

Aunque un optimizador como *AdamW* cuenta con varios hiperparámetros, el más determinante es la tasa de aprendizaje, o *learning rate*, ya que marca el tamaño de los pasos con que se ajustan los pesos en cada iteración. Un valor excesivamente alto puede provocar oscilaciones e inestabilidad, mientras que uno demasiado bajo retrasa la convergencia o incluso hace que el entrenamiento se estanque en un óptimo local. Por ello, se eligió un valor de 1×10^{-4} : lo suficientemente pequeño para garantizar un avance estable y evitar saltos muy bruscos, pero lo bastante grande para alcanzar una buena convergencia en un tiempo razonable. Esta elección se vio respaldada por los buenos resultados obtenidos en pruebas preliminares, que confirmaron su idoneidad para el modelo y datos utilizados.

4.4.2.3.- Métricas y función de pérdida

Durante la compilación del modelo también se deben especificar las métricas y la función de pérdida empleadas. A continuación se detallan las métricas y posteriormente la función de pérdida, ya que esta última deriva directamente de estas:

- **Precision:** Esta métrica cuantifica la proporción de verdaderos positivos (TP) con respecto al total de predicciones positivas (P_{pred}) realizadas por el modelo. En el contexto de segmentación binaria, la *precision* se calcula sobre la clase positiva, y permite evaluar qué tan fiables son las predicciones positivas. Se define como:

$$\text{Precision} = \frac{TP}{P_{\text{pred}}} = \frac{\sum_i y_{\text{true},i} \hat{y}_i}{\sum_i \hat{y}_i + \epsilon}. \quad (4.1)$$

Donde $y_{\text{true},i} \in \{0, 1\}$ representa la etiqueta real y $\hat{y}_i \in \{0, 1\}$ la predicción binarizada del píxel i . La binarización de las predicciones se realiza mediante redondeo, asegu-

rando que las métricas reflejen directamente el rendimiento observado. Esta métrica penaliza las falsas detecciones (falsos positivos), favoreciendo la precisión del modelo en la identificación de áreas de interés.

- **Recall:** Esta métrica evalúa la proporción de verdaderos positivos (TP) con respecto al total de positivos reales (P_{real}) presentes en las etiquetas. En tareas de segmentación binaria, la *recall* mide la sensibilidad del modelo para captar las regiones de interés. Su formulación es:

$$\text{Recall} = \frac{TP}{P_{\text{real}}} = \frac{\sum_i y_{\text{true},i} \hat{y}_i}{\sum_i y_{\text{true},i} + \epsilon}. \quad (4.2)$$

Donde $y_{\text{true},i} \in \{0, 1\}$ representa la etiqueta real y $\hat{y}_i \in \{0, 1\}$ la predicción binarizada del píxel i . Esta métrica está diseñada para evaluar que el modelo sea capaz de identificar la mayor parte posible de las regiones de interés, minimizando así las omisiones (falsos negativos).

La *precision* y el *recall* son métricas complementarias que deben considerarse juntas en lugar de por separado, ya que se enfocan en diferentes aspectos del rendimiento del modelo. Si la *precision* es alta pero el *recall* es bajo, el modelo puede estar identificando correctamente las áreas positivas, pero dejando fuera muchas otras áreas de interés, lo que resulta en una baja cobertura. Por otro lado, si el *recall* es alto pero la *precision* es baja, el modelo podría estar identificando muchas áreas como positivas, pero muchas de ellas serían falsos positivos, lo que reduce la fiabilidad de las predicciones. Al combinar ambas métricas, se busca un equilibrio entre la exactitud de las predicciones y la cobertura de las áreas relevantes, evitando los extremos donde uno de los aspectos se sacrifica en beneficio del otro.

Dado que la *precision* y el *recall* se basan en comparaciones binarias entre predicciones y etiquetas verdaderas, no son funciones diferenciables con respecto a los parámetros del modelo. Esto se debe a que implican operaciones no continuas (como la binarización por umbral), lo que genera funciones con discontinuidades y saltos abruptos. Dichas propiedades impiden el uso directo de estas métricas como funciones de pérdida dentro de marcos de optimización por gradiente. Además, mientras que las funciones de pérdida por conveniencia suelen ser minimizables, estas métricas están diseñadas para ser maximizadas, lo que introduce una incompatibilidad adicional con los algoritmos de optimización basados en gradientes. Una forma directa y efectiva de transformar estas métricas que se desean maximizar, en funciones de pérdida minimizables es emplear la operación $1 - \text{métrica}$, lo cual invierte su comportamiento de manera coherente con los objetivos del entrenamiento.

Para superar la otra limitación, se puede adoptar una reformulación diferenciable de estas métricas. En concreto, se redefinen en términos continuos utilizando directamente las salidas del modelo (no binarizadas, $\hat{y}_i \in [0, 1]$) en los cálculos, manteniendo así la capacidad

de calcular gradientes útiles para el entrenamiento. Esta aproximación permite transformar las métricas en funciones suavizadas, cuya diferenciabilidad es esencial para el aprendizaje basado en gradientes. Además, se incorpora un término de suavizado ϵ en los denominadores de las expresiones, lo cual evita divisiones por cero y asegura la estabilidad numérica durante el entrenamiento, especialmente en casos donde hay escasez de verdaderos positivos.

Sobre la base de estas métricas suavizadas y minimizables, se propone una **función de pérdida compuesta** que combina *precision* y *recall* en una sola expresión. Esta combinación permite que el modelo sea penalizado por igual ante errores de omisión (falsos negativos) o de sobresegmentación (falsos positivos), modulando su importancia relativa mediante un parámetro α . Dicho parámetro controla el peso que se asigna a cada componente de la pérdida, permitiendo ajustar el enfoque del modelo según los requerimientos específicos de la tarea. En el caso particular de este trabajo, se ha utilizado $\alpha = 0,5$, lo cual otorga igual relevancia a ambas métricas y busca un equilibrio simétrico entre sensibilidad y especificidad. La función de pérdida combinada utilizada se expresa como sigue, donde Precision y Recall corresponden a sus versiones suavizadas y diferenciables, tal como se definieron previamente, para permitir el cálculo de gradientes:

$$\mathcal{L}(y_{\text{true}}, y_{\text{pred}}, \alpha) = \alpha \cdot (1 - \text{Precision}(y_{\text{true}}, y_{\text{pred}})) + (1 - \alpha) \cdot (1 - \text{Recall}(y_{\text{true}}, y_{\text{pred}})). \quad (4.3)$$

El diseño e implementación de esta función de pérdida responde a las particularidades de las tareas de segmentación binaria, donde el objetivo principal es identificar con precisión regiones positivas en un contexto dominado por el fondo, lo que a menudo implica un cierto desbalance de clases. Ante este escenario, se observó que el uso de funciones estándar como la *Binary Cross-Entropy* (BCE) conducía a un rendimiento deficiente, con predicciones centradas casi exclusivamente en el fondo. Este comportamiento, confirmado mediante el uso de *SavePredictionsCallback*, refleja una limitación conocida de la BCE, que tiende a no penalizar adecuadamente los falsos negativos. Para superar esta deficiencia, se propuso esta función de pérdida, con el objetivo de enfocarse en la clase positiva y evaluar el rendimiento del modelo de forma más informativa. Esta reformulación no solo conserva la interpretabilidad de ambas métricas, sino que también promueve un equilibrio entre minimizar los falsos positivos y maximizar la cobertura de las regiones de interés. Su aplicación resolvió eficazmente el problema de sesgo hacia el fondo, mejorando notablemente la capacidad del modelo para segmentar las praderas de *N. noltei*. Por todo ello, se decidió adoptar esta función de pérdida como opción definitiva en el entrenamiento del modelo.

Dado que estas métricas utilizadas, aunque informativas, no siempre resultan intuitivas

para quienes no están familiarizados con los conceptos técnicos del *machine learning*, se incorporó una métrica adicional conocida como **pixel accuracy**. Esta métrica busca facilitar la interpretación de los resultados, permitiendo que cualquier persona pueda hacerse una idea clara de cuán buenas o malas son las predicciones del modelo. La *pixel accuracy* mide la proporción de píxeles correctamente clasificados sobre el total de píxeles de una imagen, comparando directamente las predicciones del modelo con las etiquetas reales. En tareas de segmentación, evalúa el grado de coincidencia entre la predicción y la realidad a nivel de píxel, representando de manera directa el porcentaje de píxeles que han sido segmentados correctamente. Esta métrica se puede expresar mediante la siguiente fórmula:

$$\text{Pixel accuracy} = \frac{\sum_{i=1}^N 1(y_{\text{true},i} = y_{\text{pred},i})}{N}. \quad (4.4)$$

Donde N es el número total de píxeles en la imagen, $y_{\text{true},i} \in \{0, 1\}$ es la etiqueta real del píxel i , y $y_{\text{pred},i} \in \{0, 1\}$ es la predicción correspondiente. La función indicadora $1(y_{\text{true},i} = y_{\text{pred},i})$ toma el valor 1 si ambos valores coinciden (es decir, el píxel fue correctamente clasificado), y 0 en caso contrario.

Tras todo este proceso, se estaría en condiciones de lanzar el entrenamiento. Cabe señalar que no se ha hecho énfasis en el número máximo de épocas ya que no es un hiperparámetro relevante debido a la estrategia de *EarlyStopping*, que detiene automáticamente el entrenamiento previamente a alcanzar el máximo de épocas.

4.5.- Implementación del módulo de inferencia

El objetivo final del entrenamiento de un modelo es su aplicación en inferencia, es decir, realizar predicciones sobre nuevas imágenes no vistas durante el proceso de entrenamiento. En esta sección se describen los pasos necesarios para procesar una nueva imagen y generar la predicción correspondiente utilizando el modelo previamente entrenado.

Cuando se recibe un nuevo ortomosaico, el preprocesado es el mismo que se describió en la sección 4.3. El primer paso consiste en reducir la resolución ya que el ortomosaico original será de una resolución muy alta. Esta reducción de resolución debe realizarse bajo las mismas condiciones que las imágenes utilizadas durante el entrenamiento. Durante este proceso, se conservan los metadatos originales de la imagen, incluyendo la transformación y el sistema de referencia de coordenadas (*CRS*), asegurando que la información geoespacial no se pierda.

Luego, se eliminan todos los canales excepto los correspondientes a las bandas *RGB*, en caso de que la imagen contenga bandas espectrales adicionales, ya que el modelo fue entrenado exclusivamente con tres bandas y no puede procesar más información. Posteriormente, se ajusta el tamaño de la imagen para que sus dimensiones sean divisibles por el tamaño del *patch*, y se divide en *patches* del mismo tamaño que los empleados durante el entrenamiento (512×512). La predicción se realiza sobre cada *patch* de manera independiente, habiendo sido previamente normalizado. Es fundamental aplicar el mismo procesamiento para evitar discrepancias que puedan afectar la precisión del modelo.

Tras la predicción, se aplica el proceso inverso: si al principio se dividió la imagen en *patches*, ahora se reconstruye la máscara a partir de los *patches* predichos por el modelo. Este proceso puede compararse con armar un *puzzle*, donde cada *patch* se coloca en la misma posición que su correspondiente en la imagen original. Finalmente, la máscara reconstruida se redimensiona al tamaño original de la imagen, asegurando que coincida exactamente con los datos iniciales. En este punto, los metadatos originales de la imagen, previamente almacenados, se copian a la máscara final, preservando la coherencia en cuanto a la transformación espacial y el sistema de referencia de coordenadas. De este modo, la máscara resultante conserva la información geoespacial de la imagen de entrada, permitiendo su integración en sistemas de análisis posteriores como *QGIS*. Además, esto posibilita la validación de la calidad de la predicción mediante el archivo *GeoJSON* conocido *groundtruth*, que se mencionó previamente en la sección relativa a los datos dentro de este mismo capítulo.

Para llevar a cabo esta validación, es necesario contar tanto con el ortomosaico original como con la máscara predicha. Ambos se cargan y se leen sus metadatos, que incluyen el tamaño de la imagen, los límites geográficos y la transformación afín. Los límites geográficos son fundamentales para realizar un *clipping* sobre los puntos del *groundtruth*, permitiendo

descartar aquellos que se encuentren fuera de estos límites.

Una vez filtrados los puntos, se procesan individualmente para extraer de ellos dos características principales: las coordenadas geográficas y la etiqueta de presencia o ausencia de *N. noltei*. Utilizando la transformación afin, estas coordenadas se convierten en índices de píxel. Tras esto, se utiliza la imagen original para descartar los puntos ubicados en zonas de relleno, es decir, fuera del área válida de la ría (véase sección 4.3 y Figura 4.8). A continuación, se emplea la máscara predicha para determinar si el píxel correspondiente es negro (0) o blanco (1).

Con esta información, se comparan las etiquetas reales obtenidas previamente del archivo *GeoJSON (groundtruth)* con los píxeles de la máscara predicha. Se contabiliza un acierto cuando un punto que indica la presencia de *N. noltei* se asocia a un píxel blanco, o cuando un punto que indica su ausencia se asocia a un píxel negro. Finalmente, el porcentaje de aciertos se calcula dividiendo el número de clasificaciones correctas entre el total de puntos evaluados y multiplicando el resultado por 100. Más técnicamente, se define N como el número total de puntos evaluados, y_i como la etiqueta real del punto i (1 para presencia y 0 para ausencia) y $\hat{y}_i$ como la etiqueta predicha (1 si el píxel es blanco y 0 si es negro). Este valor se calcula como:

$$\frac{1}{N} \sum_{i=1}^N 1(y_i = \hat{y}_i) \times 100, \quad (4.5)$$

donde $1(y_i = \hat{y}_i)$ es la función indicadora que vale 1 si la predicción es correcta y 0 en caso contrario.

5. Experimentación y resultados obtenidos

En este capítulo se presenta el trabajo experimental realizado. En primer lugar, se expondrán las especificaciones hardware de la máquina física donde se ejecutaron los experimentos, así como la arquitectura detallada de la red neuronal profunda utilizada. A continuación, se describirán los distintos experimentos llevados a cabo, explicando la motivación detrás de cada uno y el interés que pueden tener. Finalmente, se presentarán los resultados obtenidos con los diferentes enfoques, comparándolos y extrayendo conclusiones en función del rendimiento y el coste computacional.

5.1.- Especificaciones hardware de la máquina

Todos los experimentos relacionados con el entrenamiento y la evaluación del modelo se han llevado a cabo en un servidor remoto, al que se accede mediante conexión SSH. Este servidor cuenta con el sistema operativo Ubuntu 22.04.4 LTS y un procesador Intel Core i9-7900X con 10 núcleos físicos y 20 hilos, que opera a una frecuencia base de 3.30 GHz y puede alcanzar hasta 4.5 GHz. Además, dispone de 64 GiB de memoria RAM. Para las operaciones de entrenamiento del modelo, el servidor está equipado con dos GPUs NVIDIA TITAN Xp, cada una con 12 GiB de VRAM, lo que proporciona una gran capacidad para realizar operaciones con matrices y tensores de forma eficiente.

Sin las capacidades de hardware descritas, habría sido imposible llevar a cabo este proyecto, dado el gran tamaño de las imágenes procesadas y el elevado coste computacional asociado al entrenamiento del modelo, que requiere una considerable potencia de cálculo y memoria.

5.2.- Arquitectura del modelo

La arquitectura del modelo empleada en los experimentos fue una red convolucional con estructura en “U”, implementada por el equipo de Keras [24]. Se optó por este tipo de arquitectura porque, en un artículo reciente centrado en la segmentación de praderas marinas a partir de imágenes satelitales, se observó que era una de las opciones que ofrecía mejores resultados entre las evaluadas [9].

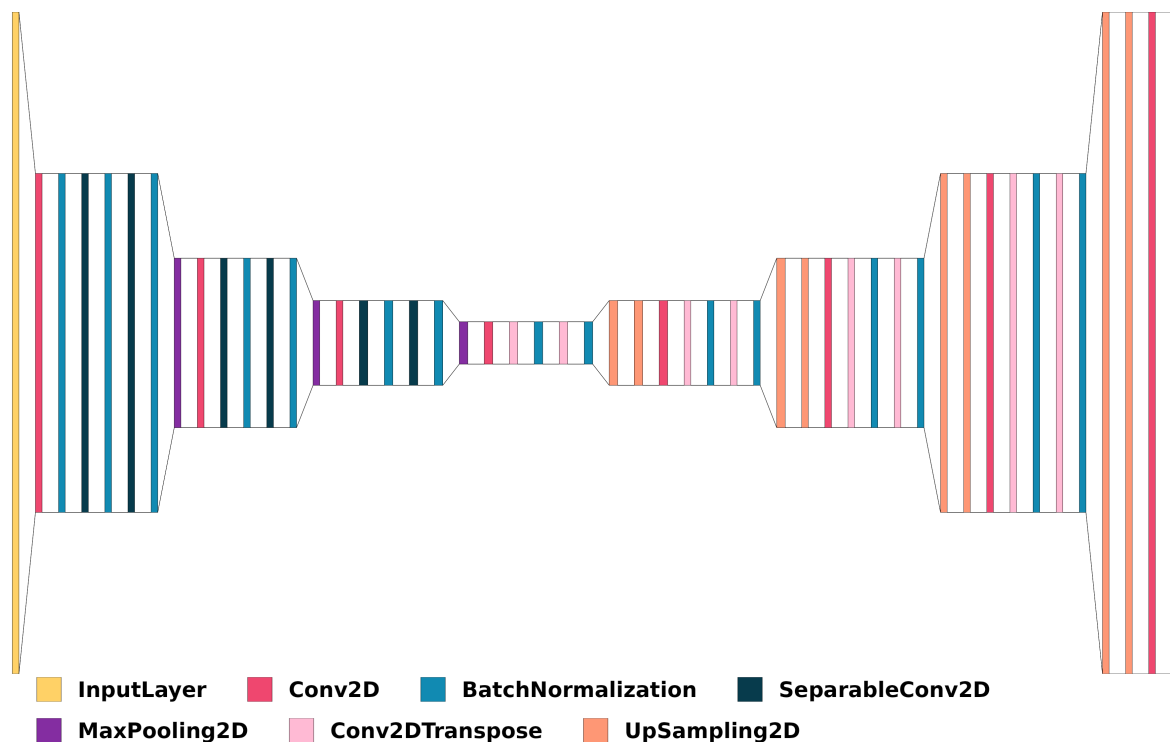


Figura 5.1: Diagrama del modelo empleado en la experimentación.

La arquitectura empleada tiene principalmente dos bloques, uno de *downsampling* y otro de *upsampling*. Esto puede apreciarse en la Figura 5.1, donde se distinguen claramente un bloque de entrada, bloques encargados de reducir y aumentar la resolución espacial.

Se hace uso de capas como convoluciones separables y convoluciones transpuestas, en lugar de convoluciones clásicas. Estas decisiones responden a una optimización práctica: las convoluciones separables permiten reducir significativamente el número de parámetros y la carga computacional sin comprometer la capacidad de extracción de características, lo cual resulta especialmente relevante en el contexto de este trabajo, donde se trabaja con imágenes de alta resolución y recursos computacionales limitados. Por otro lado, las convoluciones transpuestas y las operaciones explícitas de *upsampling* ofrecen mayor control sobre la recuperación espacial de la información, ayudando a reconstruir con precisión los contornos de objetos pequeños, como pueden ser algunas regiones concretas de *N. noltei*.

En la etapa de *downsampling*, se inicia con un bloque de entrada que aplica una convolución 3×3 con un paso (*stride*) de 2 y *padding*, seguida de normalización por lotes (*batch normalization*) y activación *ReLU*. Este bloque inicial utiliza 32 filtros. A continuación, se incorporan tres bloques consecutivos que aumentan progresivamente la cantidad de filtros a 64, 128 y 256, respectivamente. En cada uno de estos tres bloques, los mapas de características pasan por dos secuencias de activación *ReLU*, convolución separable 3×3 y normalización por lotes. Para reducir la resolución espacial, se aplica una operación de *max-pooling* 3×3 con *stride* de 2. Además, en cada bloque se incorpora una conexión residual que proyecta la salida del bloque anterior mediante una convolución 1×1 (con *stride* de 2 y *padding* para ajustar la dimensión) y se suma a los mapas de características actuales. Esto ayuda a preservar información relevante y a mejorar el flujo de gradientes durante el entrenamiento.

La etapa de *upsampling* invierte el proceso anterior para recuperar la resolución espacial de las características. En esta fase se emplean cuatro bloques con un número decreciente de filtros: 256, 128, 64 y 32, respectivamente. Cada bloque aplica dos secuencias de activación *ReLU*, convolución transpuesta 3×3 y normalización por lotes, seguidas de una capa de *upsampling* 2×2 . Además, se incluye una conexión residual que toma la salida del bloque anterior, la amplía mediante *upsampling* 2×2 y aplica una convolución 1×1 para alinearla con los mapas de características actuales. Esta combinación, ilustrada en la Figura 5.1, mejora la coherencia de la reconstrucción y facilita la recuperación de detalles finos.

Finalmente, la red concluye con una capa de salida que aplica una convolución 3×3 con activación sigmoide, generando un único mapa de salida en el que cada píxel representa la probabilidad de pertenecer a la clase de interés.

Esta red fue entrenada siguiendo la metodología y configuración del *pipeline* de entrenamiento descrita en el capítulo 4. No obstante, con el objetivo de facilitar la comprensión y servir como recordatorio, se incluye a continuación una tabla resumen con los hiperparámetros establecidos. Aquellos hiperparámetros con mayor impacto en el rendimiento del modelo, como la tasa de aprendizaje o el tamaño del *batch*, fueron seleccionados mediante un proceso preliminar de prueba y ajuste, evaluando entre dos y tres de los valores más habituales y eligiendo aquellos que ofrecían resultados más consistentes. En el caso del resto de los hiperparámetros, se mantuvieron, en la mayoría de los casos, los valores por defecto, siempre que se consideraron adecuados para el problema abordado. Cabe señalar que estos hiperparámetros sirven como la base sobre la que se llevaron a cabo los experimentos, asegurando un marco equitativo que permita la comparación adecuada de los resultados obtenidos con los distintos enfoques.

Hiperparámetro	Valor
Tamaño del batch (<i>batch size</i>)	32
Paciencia del <i>early stopping</i>	30
Mejora mínima (<i>min_delta</i>)	0.000001
Factor de reducción de <i>ReduceLROnPlateau</i>	0.1
Paciencia de <i>ReduceLROnPlateau</i>	10
Tasa de aprendizaje mínima (<i>min_lr</i>)	10^{-6}
Optimizador	AdamW
Tasa de aprendizaje inicial (<i>learning rate</i>)	1×10^{-4}
Función de pérdida (<i>loss</i>)	<i>Combined loss</i> 4.3
Ponderación de combined loss (<i>alpha</i>)	0.5
Métricas	<i>Precision</i> 4.1 , <i>Recall</i> 4.2 , <i>Pixel accuracy</i> 4.4
Número máximo de épocas	2000

Tabla 5.1: Hiperparámetros empleados durante el entrenamiento del modelo.

5.3.- Evaluación del impacto del filtrado de imágenes y máscaras sin regiones de interés

Durante la carga de los datos se puede decidir si se quiere entrenar el modelo con todos los *patches* obtenidos tras el preprocesamiento o si se desea un paso de filtrado adicional que elimina los *patches* que no contienen *N. noltei*, ya que existe un gran número de este tipo y se ha observado en las pruebas iniciales que se obtienen resultados relativamente similares entrenando de una u otra forma. Sin embargo, es importante evaluar de forma exhaustiva si este filtrado tiene algún impacto sobre el rendimiento del modelo, es decir, si al eliminar los *patches* sin regiones de interés se logra un desempeño igual o ligeramente inferior, o si por el contrario esta eliminación afecta negativamente a la capacidad del modelo para generalizar, reduciendo su eficacia en la segmentación de las áreas de interés.

Evaluar el impacto de este filtrado es relevante ya que si el filtrado resulta beneficioso, supondría varias ventajas: un entrenamiento más eficiente al reducir el tamaño del conjunto de datos, una menor carga computacional y una convergencia más rápida del modelo, ya que se evitaría procesar imágenes sin contenido útil. Además, eliminar datos irrelevantes podría minimizar el riesgo de sesgo y ayudar al modelo a centrarse en aprender patrones significativos, lo que podría traducirse en una mejor detección de las regiones de interés. Por el contrario, si se pierde rendimiento en inferencia habrá que cuantificar la pérdida para poder evaluar minuciosamente si merece la pena asumirla a cambio de estos beneficios.

A continuación, se describirá el desarrollo del experimento, en el que se entrenaron dos modelos: uno utilizando el conjunto de datos completo y otro con el conjunto filtrado. Además se comentarán los resultados obtenidos, finalizando con una discusión sobre las conclusiones extraídas de la comparación entre ambas configuraciones.

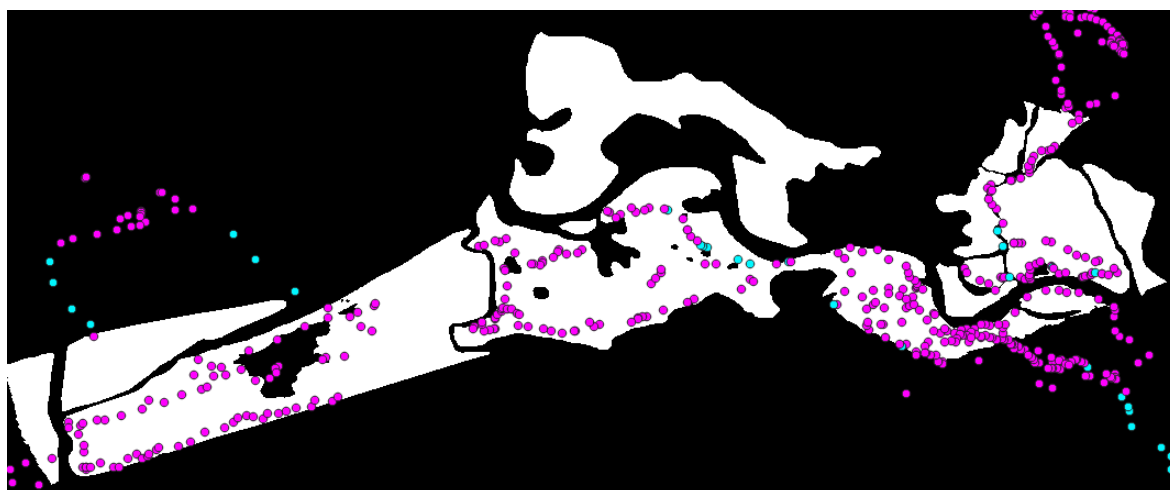
Cabe mencionar que previamente a la división en *patches* y por ende, del filtrado, se separó un ortomosaico completo sobre el que se seguirá el método de inferencia explicado en el capítulo previo. Esto se ha hecho con el objetivo de evaluar el modelo en producción, es decir, para un ortomosaico completo nuevo, medir si modelo es capaz de generalizar correctamente y detectar las zonas de *N. Noltei*. La imagen y máscara elegida como validación es la que se muestra en la Figura 5.2. Se decidió utilizar esta ya que como se ve, existe una representación bastante homogénea de puntos de *groundtruth*, a pesar de que la gran mayoría son de presencia.

Como ya se dijo, las máscaras están hechas a groso modo, sin detalle, lo que ocasiona que la máscara real obtenga un porcentaje de acierto respecto a los puntos de *ground truth* del 91.74 %. De hecho, en la Figura 5.2b puede verse como dentro de los límites del ortomo-

saico real algunos puntos clasificados como presencia están ubicados sobre píxeles negros, clasificados erróneamente por la máscara.



(a) Ortomosaico original



(b) Máscara binaria

Figura 5.2: Ortomosaico empleado para validación junto con su máscara binaria. Los puntos de *ground truth* indican presencia (rosa) o ausencia (azul) de *N. Noltei*.

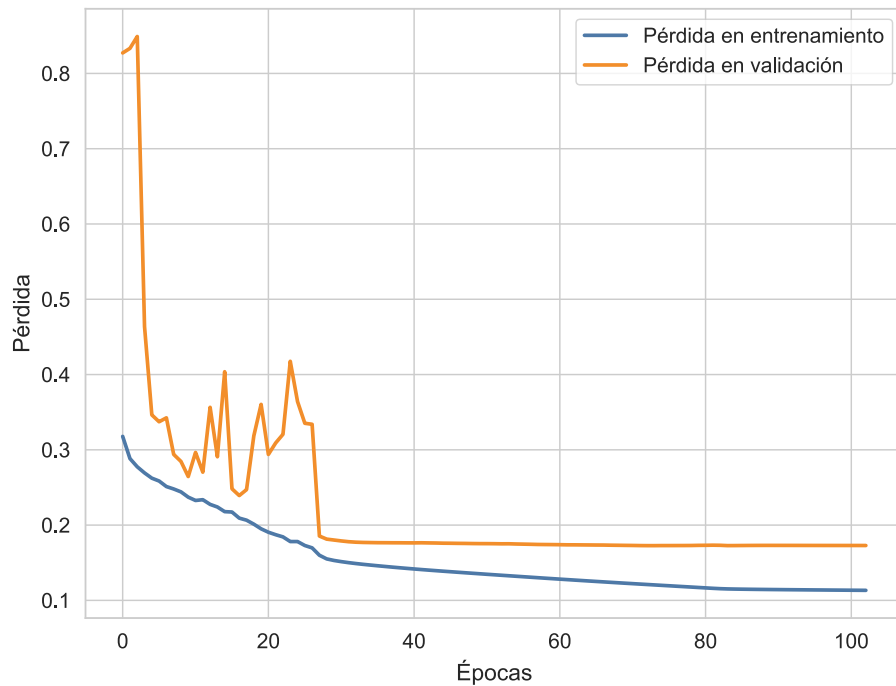
5.3.1.- Ajuste del modelo con el conjunto de datos original

Primeramente se llevó a cabo el entrenamiento del modelo con todos los datos obtenidos tras el preprocesado de estos, siguiendo la configuración del *pipeline* (véase capítulo 4). Los hiperparámetros son los vistos en la Tabla 5.1 y la distribución de los datos en los diferentes conjuntos se puede ver en la Tabla 4.3.

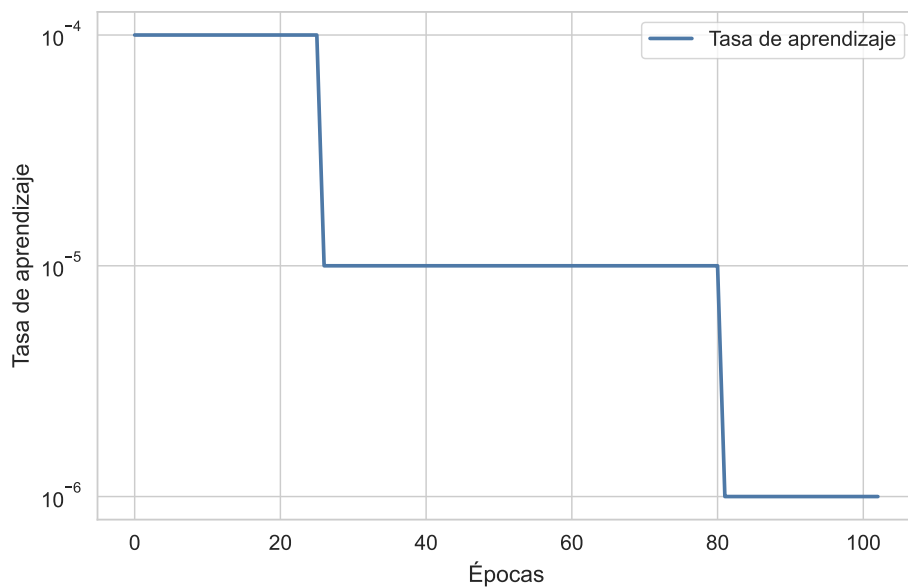
El entrenamiento se completó tras un total de 103 épocas, donde cada época tarda apro-

ximadamente 2 minutos en procesarse lo que da lugar a un tiempo de entrenamiento de 2 horas y 46 minutos. Como se puede ver en la Figura 5.3a, el error en entrenamiento desde un principio fue relativamente bajo y fue descendiendo de manera muy estable hasta aproximadamente la época 30, donde empieza a descender muy lentamente, hasta que a partir de la época 80, descende de manera prácticamente imperceptible debido a que, como se ve en la Figura 5.3b, el *learning rate* descende hasta un número lo suficiente pequeño como para permitir que el entrenamiento siga evolucionando aunque a un ritmo menor. Esto se debe a que se definió un *callback* que tras 10 épocas sin mejora disminuye el valor de este hiperparámetro.

En cuanto al error en validación, como es de esperar empieza siendo notablemente alto al principio. Además, debido a un mayor valor del *learning rate* se puede observar como en las épocas iniciales el entrenamiento es algo más inestable, existiendo épocas con una pérdida muy baja y otras más alta. No es hasta que, entre la época 20 y 40, disminuye el valor del *learning rate* y entonces el error también disminuye considerablemente, llegando por debajo de 0.2 y tiende a estabilizarse hasta que, como sucedía en la pérdida del conjunto de entrenamiento, empieza a descender tan lentamente que no se puede percibir en la imagen, llegando a estancarse y tras 30 épocas sin mejorar, se detiene. La mejor época fue la 73, con una pérdida de 0.17272 en validación.



(a) Evolución del error durante el entrenamiento con el conjunto de datos original



(b) Evolución del *learning rate* durante el entrenamiento con el conjunto de datos original

Figura 5.3: Comparación de la evolución del error y del *learning rate* durante el entrenamiento

Finalmente, con el mejor modelo obtenido durante el entrenamiento se hace inferencia con un conjunto de *patches* que se separaron el objetivo de evaluar el modelo. Los resultados

en test fueron los que se observan en la Tabla 5.2. Como se puede ver, el modelo generaliza notablemente bien ya que el error no difiere en gran medida al obtenido en validación y acierta el 86 % de los píxeles predichos.

Métrica	Valor
Loss	0.1824
Positive Precision	0.7329
Positive Recall	0.9030
Pixel Accuracy	0.8676

Tabla 5.2: Resultados en el conjunto de test empleando el modelo entrenado con el conjunto de datos original

Aún observando que en términos de error y métricas de entrenamiento aparentemente es un buen modelo, lo que realmente importa es que en producción, es decir, que ante un ortomosaico nuevo, el modelo sea capaz de generalizar correctamente y detectar las zonas de *N. Noltei*. La Figura 5.4 muestra la máscara predicha por el modelo para el ortomosaico de validación, que puede verse para su comparación en la Figura 5.2. Se puede ver como el modelo es capaz de diferenciar perfectamente lo que es el cauce principal de la ría y por lo tanto sin presencia de *N. Noltei*. En cuanto a las zonas que indican presencia, se puede ver como hay regiones donde el modelo está muy seguro de que su presencia y prácticamente todos los píxeles de la región son blancos, formando unas zonas densas. Sin embargo existen zonas más complejas que el modelo no es capaz de discernir con alta confianza si se trata de pradera marina u otro elemento y ante la duda genera pequeños artefactos o 'islas' de píxeles blancos.

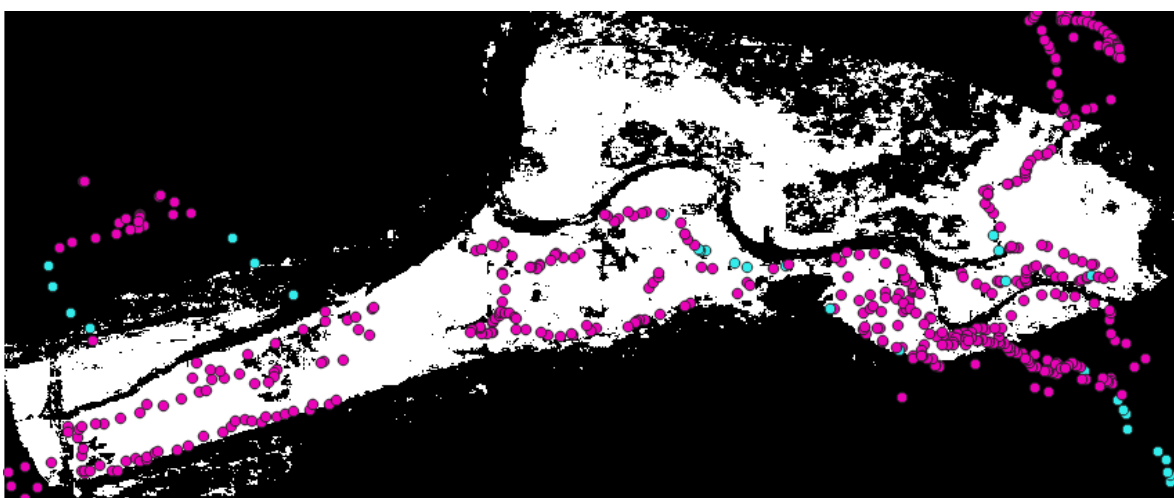


Figura 5.4: Máscara predicha por el modelo entrenado con el conjunto original. Superpuestos, los puntos de *ground truth* que indican presencia (rosa) o ausencia (azul)

Como se tiene la máscara predicha para un ortomosaico completo, se puede emplear la métrica 4.5 definida en el capítulo previo. A modo de resumen, esta métrica otorgaba un porcentaje de acierto en base a los puntos georreferenciados que sirven de *ground truth*. Para esta métrica el modelo obtiene un porcentaje de acierto del 89.09 %, lo cual es bastante cercano al valor de la máscara real. Se estudió más a fondo las predicciones del modelo, construyendo una matriz de confusión (véase Figura 5.5). Se puede observar cómo el modelo acierta un total de 298 puntos positivos, mientras que falla 24 puntos que clasifica como positivos cuando realmente son negativos. Aún habiendo una baja proporción de puntos negativos, el modelo solo acierta 4, fallando 13 puntos que clasifica como negativos cuando realmente son positivos.

Valor real	Positivo real	298	13
	Negativo real	24	4
		Positivo predicho	Negativo predicho
		Predicción	

Figura 5.5: Matriz de confusión del modelo entrenado a partir del conjunto original

5.3.2.- Ajuste del modelo con el conjunto de datos filtrado

Para llevar a cabo este segundo entrenamiento se filtró el conjunto de datos original, eliminando aquel par de *patches* imagen-máscara cuya máscara tuviese un porcentaje de píxeles con valor igual a 0 superior al 95 %, esto es que no tuviese prácticamente ninguna región de interés. Tras hacer esto, la división en los distintos conjuntos de datos empleados durante el entrenamiento quedó como se ve en la Tabla 5.3. El entrenamiento del modelo se llevó a cabo con la configuración del *pipeline* explicado en el capítulo 4 y los hiperparámetros son los vistos en la Tabla 5.1, misma configuración que en el caso previo.

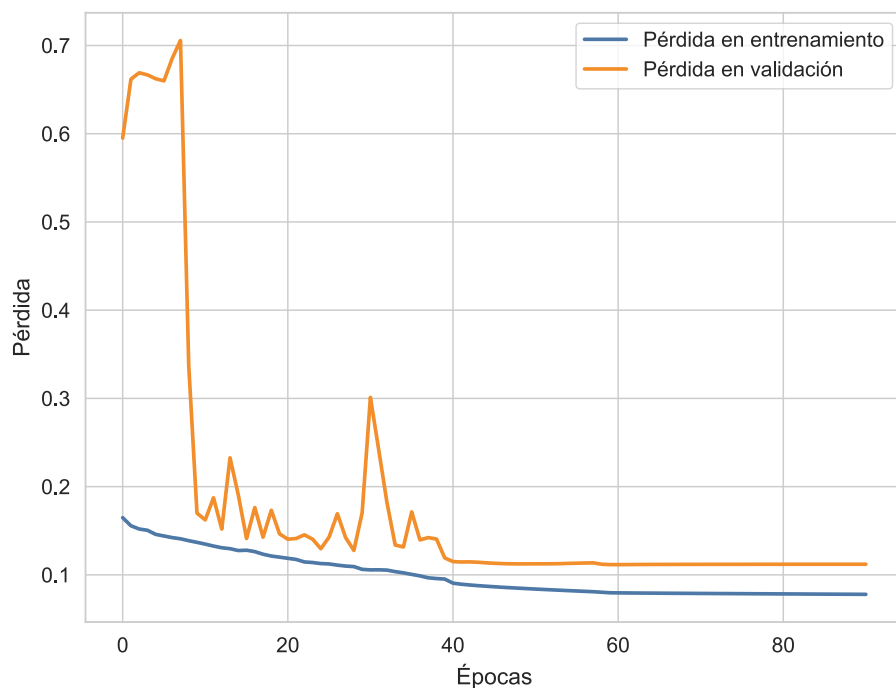
Conjunto	Porcentaje	Cantidad de <i>patches</i>
Entrenamiento	70 %	2144
Validación	20 %	613
Test	10 %	307
Total	100 %	3064

Tabla 5.3: Distribución de imágenes por conjunto una vez filtrado

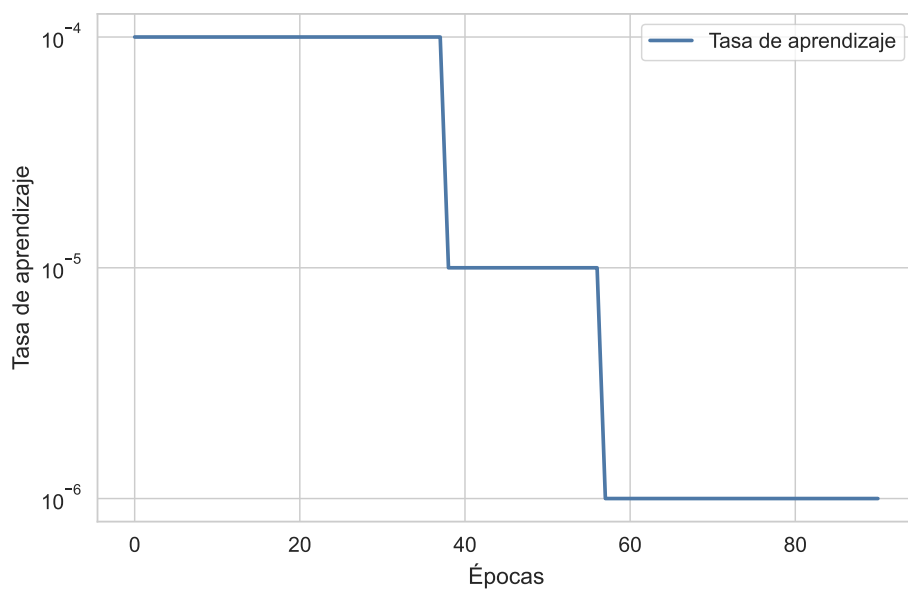
Esto es una disminución del 52.1 %, es decir, el conjunto se ha reducido prácticamente a la mitad. Esta es una reducción bastante considerable, de ahí que sea relevante evaluar exhaustivamente el rendimiento del modelo con ambas configuraciones. Por un lado, debido a que reducir a la mitad el tamaño del conjunto puede traer ciertos beneficios como los mencionados en la introducción de este experimento. Por otro lado, debido a que al ser una reducción tan drástica no es posible emplearla a la ligera, sino que se debe comprobar que no afecta al rendimiento del modelo.

Como se puede ver en la Figura 5.6a, la evolución del error es similar al caso anterior, aunque generalmente más bajo tanto en entrenamiento como en validación. Para el conjunto de entrenamiento comienza en un valor muy bajo y rápidamente se estabiliza y comienza a descender pausadamente debido al valor del *learning rate*. Para validación también se asemeja al caso previo. Al principio comienza bastante inestable con una gran subida y luego una gran bajada, pero rápidamente cae por debajo de 0.2. Al contrario de lo que ocurre con el error de entrenamiento, este error permanece bastante más inestable, llegando por momentos a oscilar notoriamente, probablemente debido a un elevado valor de *learning rate*. No es hasta que este disminuye, como se ve en la Figura 5.6b, donde el error se estabiliza y comienza a descender lentamente, rozando un valor de 0.1, hasta que en la época 91 se detiene debido a la parada temprana. En este caso la mejor época ha sido la 61 con un error de validación de 0.11159.

Como se comentó en la introducción, el coste computacional y el tiempo requerido para ajustar un modelo con estos datos es menor. El entrenamiento se completó en 91 épocas, lo que conllevó un total de 1 hora y 11 minutos, ya que cada época se procesa en aproximadamente 50 segundos. Esto representa una reducción del tiempo total de entrenamiento de aproximadamente un 57 % con respecto al entrenamiento anterior (2 horas y 46 minutos).



(a) Evolución del error durante el entrenamiento con el conjunto de datos filtrado



(b) Evolución del *learning rate* durante el entrenamiento con el conjunto de datos filtrado

Figura 5.6: Comparación de la evolución del error y del *learning rate* durante el entrenamiento con el conjunto de datos filtrado

Como era esperable a partir de los resultados del entrenamiento, este modelo presenta una reducción significativa del error sobre los *patches* de test, tal como se muestra en la Tabla

5.4. Obtiene una pérdida considerablemente menor, junto con un aumento en el *recall* y la *precisión*. No obstante, a nivel de píxel, el porcentaje de acierto es algo inferior, alcanzando casi un 83 %.

Métrica	Valor
Loss	0.1280
Positive Precision	0.8215
Positive Recall	0.9229
Pixel Accuracy	0.8295

Tabla 5.4: Resultados en el conjunto de test empleando el modelo entrenado con el conjunto de datos filtrado

A continuación se muestra la máscara predicha para el ortomosaico de validación 5.2 por este modelo:

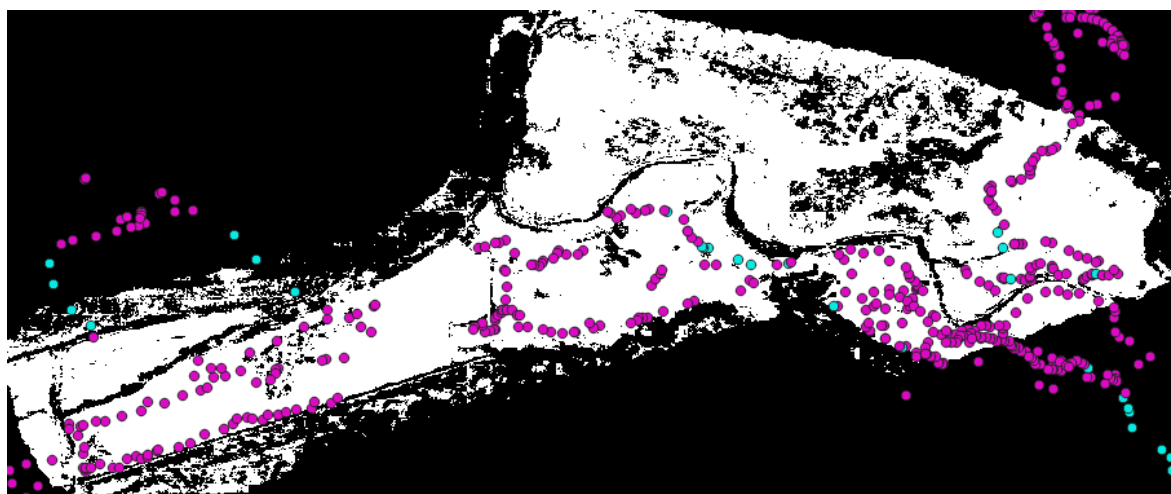


Figura 5.7: Máscara predicha por el modelo entrenado con el conjunto filtrado. Superpuestos, los puntos de *ground truth* que indican presencia (rosa) o ausencia (azul)

Si se compara la máscara real con la predicha a partir de este modelo, se puede observar como el modelo tiende a predecir mayor cantidad de regiones de interés, tiene tendencia a clasificar mayor cantidad de píxeles como 1. Sin embargo, en aquellas zonas con presencia de *N. Noltei*, el modelo crea una mayor densidad de zonas blancas, consiguiendo una segmentación más continua y homogénea. Esto se contrasta con la métrica de validación 4.5 y los puntos de *ground truth*, ya que se obtiene un valor del 92.04 %, lo cual es un valor notablemente alto, incluso superior al de la propia máscara real.

De hecho, como se observa en la matriz de confusión de la Figura 5.8, el modelo acierta un total de 309 puntos positivos, clasificando solo 13 puntos que realmente son negativos como positivos. Respecto a los puntos negativos, el modelo acierta correctamente 3, clasificando erróneamente 14 puntos como negativos cuando realmente son positivos. Como ya se notó, el modelo predice mayor cantidad de regiones de interés y por ende acierta más puntos de validación que indican presencia.

Valor real	Positivo real	309	14
	Negativo real	13	3
		Positivo predicho	Negativo predicho
		Predicción	

Figura 5.8: Matriz de confusión del modelo entrenado a partir del conjunto filtrado

5.3.3.- Discusión de los resultados

Los resultados obtenidos a partir de ambos modelos son en general muy similares. Si se desglosa esta conclusión un poco más, se han empleado dos formas de evaluación; por un lado las que se pueden denominar 'clásicas', esto es el error y las métricas para evaluación; y por otro lado la métrica personalizada que se definió para validación de la predicción de un ortomosaico completo a partir de los puntos de *ground truth*.

En base a la primera vertiente de evaluación los resultados son extremadamente similares. El modelo entrenado a partir del conjunto de datos filtrado obtiene mayores valores tanto para *recall* como para *precision*, donde en esta última métrica lo supera en casi 0.1. Además, consigue una pérdida tanto en validación como en test menor, de alrededor de un 0.05. Sin embargo, en cuanto a acierto a nivel de píxel, este modelo es aproximadamente 0.03 inferior respecto al modelo entrenado con todos los datos disponibles. Esta conclusión se puede extraer simplemente observando las tablas 5.2 y 5.4.

Cabe matizar en este punto que, aunque se incluyen métricas clásicas, es importante señalar que los modelos comparados han sido entrenados con configuraciones distintas de datos de entrada, lo cual impide una comparación directa y completamente justa basada únicamente en estas métricas. Aun así, estas medidas resultan útiles para obtener una impresión general del comportamiento de cada modelo, y pueden ayudar a detectar patrones de error sistemáticos. En este caso, las diferencias entre las métricas obtenidas son mínimas, lo que sugiere que el rendimiento cuantitativo es similar entre configuraciones. Por tanto, se incluyen las métricas como complemento al análisis visual y a la validación sobre ortomosaicos completos, que se considera más representativa del rendimiento real del sistema en su aplicación final.

Si se comparan los resultados en base a la métrica personalizada definida en el apartado 4.5, el modelo entrenado con el conjunto filtrado supera al modelo entrenado a partir del conjunto completo en un valor de 0.03. Como se puede ver en las matrices de confusión (Figuras 5.5 y 5.8), la segunda, procedente del modelo entrenado a partir del conjunto filtrado, obtiene mejores valores, ya que acierta 11 puntos de presencia más. Esto se debe a que el modelo tiende a predecir mayor cantidad de zonas de interés como se observa en la propia máscara predicha (Figura 5.7). A pesar de esto, el modelo solo falla un punto más de ausencia que su contraparte.

Aquí surge un pequeño dilema y es que, por un lado, las máscaras 'reales' no son del todo fiables debido a que, como ya se dijo, están hechas sin detallar contornos, bordes o zonas complicadas debido al elevado tiempo que conllevaría hacer esto. Por lo tanto, es posible que esto esté limitando en cierta medida la capacidad de aprender del modelo, ya que puede ser que reconozca zonas de presencia de *N. Noltei* que sin embargo están sin marcar en la máscara real. Es decir, es posible que algunas predicciones del modelo sean realmente correctas a pesar de que la máscara real diga lo contrario, o viceversa. Por otro lado, los puntos de *ground truth* sí que son de alta fiabilidad. El problema que existe es la baja distribución de puntos que indican ausencia, lo que ocasiona que la evaluación esté ligeramente sesgada ya que no es posible evaluar correctamente si el modelo produce falsos positivos, es decir, si tiende a predecir regiones de presencia cuando realmente no lo son.

Tras presentar los resultados con ambas vertientes de evaluación y analizar las ventajas e inconvenientes de cada una, queda claro que no es sencillo determinar de forma definitiva qué configuración ofrece un rendimiento superior. Las métricas tradicionales reflejan un desempeño muy similar entre los dos modelos, y aunque la métrica personalizada muestra una ligera preferencia por el modelo entrenado con el conjunto filtrado, dicha diferencia no es lo suficientemente grande para que sea concluyente. Sin embargo, se otorga prioridad a esta métrica personalizada, ya que representa de manera más fiel la realidad del problema, incluso considerando un posible sesgo.

Dado que no existen indicios de que el modelo con datos filtrados sea inferior y que además aporta ventajas computacionales significativas, la decisión es optar por utilizar el conjunto filtrado en las pruebas futuras, asegurando así un equilibrio entre precisión y eficiencia.

5.4.- Evaluación del rendimiento con ortomosaicos a distintas resoluciones

Este experimento tiene como objetivo evaluar el impacto de la resolución de los ortomosaicos en el rendimiento del modelo. Como se ha comentado en el capítulo previo, las imágenes originales se remuestran a una resolución más baja mediante un factor de resolución para su posterior división en *patches* y uso en el entrenamiento del modelo. Estos *patches* siempre tienen el mismo tamaño, pero la cantidad de detalles que contienen varía según la resolución de la imagen original. A mayor resolución, cada píxel representa un área más pequeña, capturando más detalles del terreno. En cambio, con menor resolución, cada *patch* contiene menos detalle debido a que cada píxel abarca una superficie mayor.

Este análisis es particularmente interesante porque la resolución espacial de las imágenes está directamente relacionada con la altura a la que se debe volar el dron para capturarlas. Cuando el dron vuela a una menor altura, los píxeles cubren un área más pequeña del terreno, lo que resulta en una mayor cantidad de detalle del terreno. En cambio, si el dron vuela a una mayor altura, los píxeles abarcan un área más extensa, lo que reduce la cantidad de detalle. La elección de la altura a la que vuela el dron no solo afecta la resolución, sino también la eficiencia del mapeo. A mayor altura, cada imagen cubre un área más grande, lo que permite capturar la totalidad de la región en menos pasadas y, por ende, reducir el tiempo total de vuelo. Sin embargo, esto implica una menor resolución y la posible pérdida de detalles finos. Por otro lado, volar a una menor altura genera imágenes de mayor resolución, pero requiere más tiempo para completar el mapeo, ya que se necesitan más capturas para cubrir la misma superficie. Por todos estos motivos, es fundamental comprender cómo la resolución afecta al rendimiento del modelo, ya que esto condicionará las decisiones sobre la toma de imágenes con el dron para obtener resultados óptimos.

El procedimiento experimental consistió en entrenar y evaluar el rendimiento de dos modelos con arquitecturas idénticas, diferenciados únicamente por la resolución de los datos con los que fueron entrenados. El primero se entrenó con imágenes de mayor resolución, donde cada píxel representa 0.05 metros en el terreno (resolución angular de 0.0000006°); mientras que el segundo se entrenó con imágenes de menor resolución, en las que cada píxel equivale a 0.1 metros (resolución angular de 0.0000012°), lo que implica una reducción del 50 % en la resolución espacial.

Al igual que en el experimento anterior, se dejó fuera del entrenamiento un ortomosaico completo (véase la Figura 5.2), el cual se utilizó como conjunto de validación para ambos modelos. Este ortomosaico fue adaptado a la resolución correspondiente en cada caso, de modo que se garantizara una evaluación coherente con las condiciones de entrenamiento de

cada modelo.

Nombre de la imagen	Dimensiones (px)	Tamaño (MB)
Encienona.tif	4224×4443	31.81
Misiego.tif	3791×5118	31.93
Bornizal.tif	4235×3383	29.63
Bornizal2.tif	5473×8253	98.85
Izq1.tif	6481×10558	76.91
Molino1.tif	10886×6457	86.08
Molino2.tif	10291×9792	182.25
Izq2.tif	18376×10387	249.00
Izq3.tif	10926×12059	165.87
Izq4.tif	2747×8925	55.13
Dcha_remate1.tif	4468×3682	25.32
Dcha_remate2.tif	9134×11981	173.38
Dcha_remate3.tif	5383×4601	45.32
Albatros_nanoref.tif	2641×3125	13.32

Tabla 5.5: Características de los ortomosaicos tras aplicar la reducción más drástica de resolución (0.1 metros/píxel).

La Tabla 5.5 presenta las dimensiones y tamaños de los ortomosaicos tras aplicar un remuestreo adicional que reduce aún más la resolución espacial de las imágenes. En comparación con las dimensiones originales (véase Tabla 4.1) y con la primera versión remuestreada (Tabla 4.2), esta nueva reducción permite disminuir de forma considerable el tamaño de los ortomosaicos, lo que resulta especialmente útil para agilizar el procesamiento durante el entrenamiento.

5.4.1.- Ajuste del modelo con ortomosaicos de mayor resolución (0.05 m/píxel)

El entrenamiento del modelo se llevó a cabo con los datos filtrados, de acuerdo con los resultados del experimento previo. La configuración del *pipeline* se mantiene como se explicó en el capítulo 4, con los hiperparámetros especificados en la Tabla 5.1 y la distribución de datos detallada en la Tabla 5.3.

Dado que la resolución empleada en este apartado es de 0.0000006° (equivalente a 0.05 metros por píxel), los resultados obtenidos son exactamente los mismos que los reportados en la segunda parte del experimento previo (5.3), donde esta resolución fue utilizada como referencia. Para evitar redundancias y hacer el documento más claro y conciso, no se vuelven

a incluir las gráficas ni a repetir el análisis de resultados. Estos pueden consultarse en la subsección 5.3.2. No obstante, en la subsección 5.4.3, dedicada a la discusión, se hará un breve repaso de estos resultados con el fin de facilitar la comparación entre ambos enfoques.

5.4.2.- Ajuste del modelo con ortomosaicos de menor resolución (0.10 m/píxel)

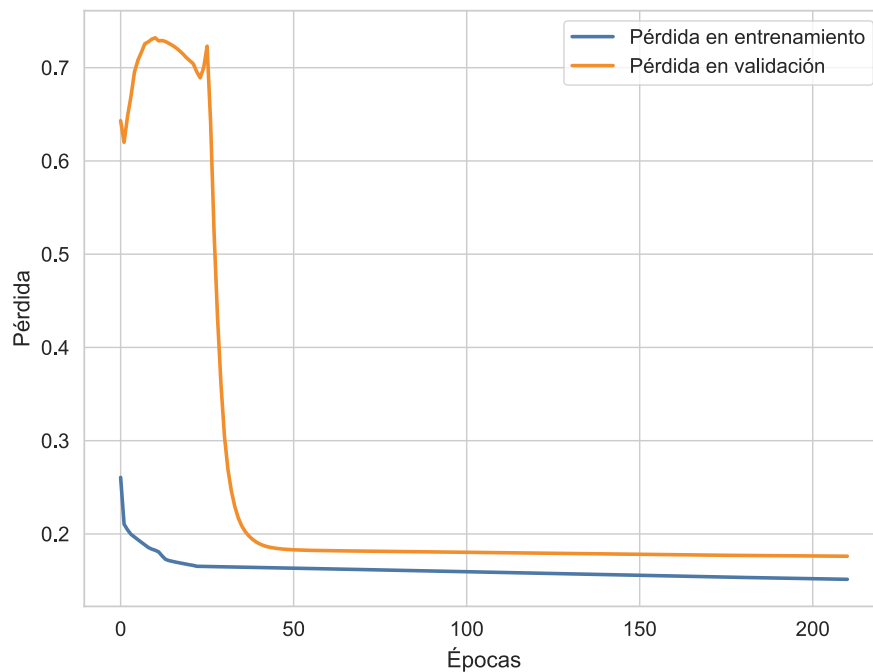
Del mismo modo que en el enfoque anterior, se llevó a cabo el entrenamiento del modelo con los datos filtrados y con la configuración del *pipeline* explicado en el capítulo 4. Los hiperparámetros son los vistos en la Tabla 5.1 y la distribución de los datos en los diferentes conjuntos se puede ver en la Tabla 5.6. En este caso, se obtiene un número de *patches* bastante bajo, debido en parte al filtrado, pero eso es algo que también sucedía en el caso previo y se tiene un número generoso de estos. La principal razón por la que existe esa gran disminución de datos es debido a la resolución, que se disminuyó en un 50 %, lo que ocasiona que el tamaño de la imagen se vea reducido, y por ende, al dividir en *patches* de tamaño fijo (512x512), se obtienen menos, aproximadamente un 66 % menos.

Conjunto	Porcentaje	Cantidad de <i>patches</i>
Entrenamiento	70 %	717
Validación	20 %	206
Test	10 %	103
Total	100 %	1026

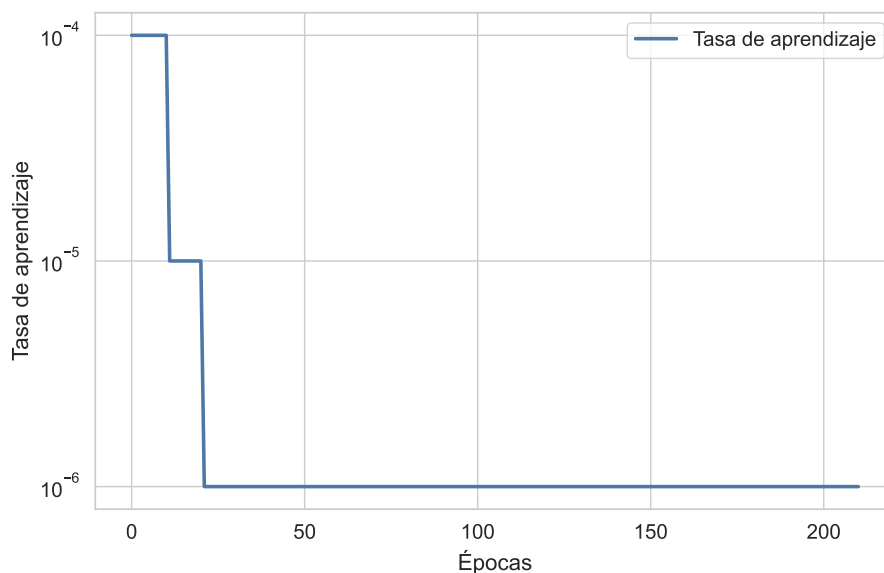
Tabla 5.6: Distribución de imágenes por conjunto una vez filtrado

El entrenamiento se completó tras un total de 211 épocas, donde cada una se procesa en aproximadamente 15 segundos, lo que da un tiempo total de entrenamiento de 53 minutos. Como era de esperar el tiempo de ejecución de cada época y por ende el entrenamiento completo es considerablemente bajo ya que durante cada época se procesan muchos menos datos.

La Figura 5.9a muestra la evolución del error. Como se ve, el error para el conjunto de entrenamiento es bajo y estable desde el principio. De hecho, a partir de la época 50 se vuelve imperceptible la disminución de este error. Lo mismo ocurre para el error de validación, con la pequeña diferencia de que al inicio el error se estanca y permanece elevado durante varias épocas, incluso cuando desciende el *learning rate* por primera vez (Figura 5.9b), ya que no es hasta que este disminuye por segunda vez que el error se estabiliza y comienza a descender paulatinamente hasta superar las 30 épocas sin mejorar que es cuando se detiene el entrenamiento. La mejor época fue la 181 con una pérdida en validación de 0.17507.



(a) Evolución del error durante el entrenamiento del modelo entrenado con las imágenes a menor resolución



(b) Evolución del *learning rate* durante el entrenamiento del modelo entrenado con las imágenes a menor resolución

Figura 5.9: Comparación de la evolución del error y del *learning rate* durante el entrenamiento con imágenes a menor resolución

Finalmente con el mejor modelo obtenido durante el entrenamiento se hace inferencia con el conjunto de *patches* que se reservó para test. Los resultados obtenidos con este modelo pueden verse en la Tabla 5.7. Las métricas son en general más pobres que en los casos estudiados previamente. El error ronda el 0.2, *precision* está por debajo 0.7 y el porcentaje de acierto a nivel de píxel apenas llega al 75 %. En lo único que destaca el modelo es en un alto valor de *recall*. Sin embargo, un alto valor en esta métrica pero que no se acompaña de un alto valor de *precision* es algo negativo ya que indica que el modelo está identificando muchas regiones como positivas, pero con una alta tasa de falsos positivos. Esto sugiere que la segmentación es menos precisa y que el modelo tiende a sobresegmentar, detectando presencia donde no la hay.

Métrica	Valor
Loss	0.1942
Positive Precision	0.6788
Positive Recall	0.9340
Pixel Accuracy	0.7483

Tabla 5.7: Resultados en el conjunto de test empleando el modelo entrenado con las imágenes a menor resolución

A continuación se analizará cómo se comporta el modelo ante un ortomosaico completo en lugar de solo con *patches* de test, siguiendo la metodología de inferencia definida en capítulo 4. La Figura 5.10 muestra la máscara predicha por el modelo para el ortomosaico de validación, que puede verse para su comparación junto a la máscara real en la Figura 5.2.

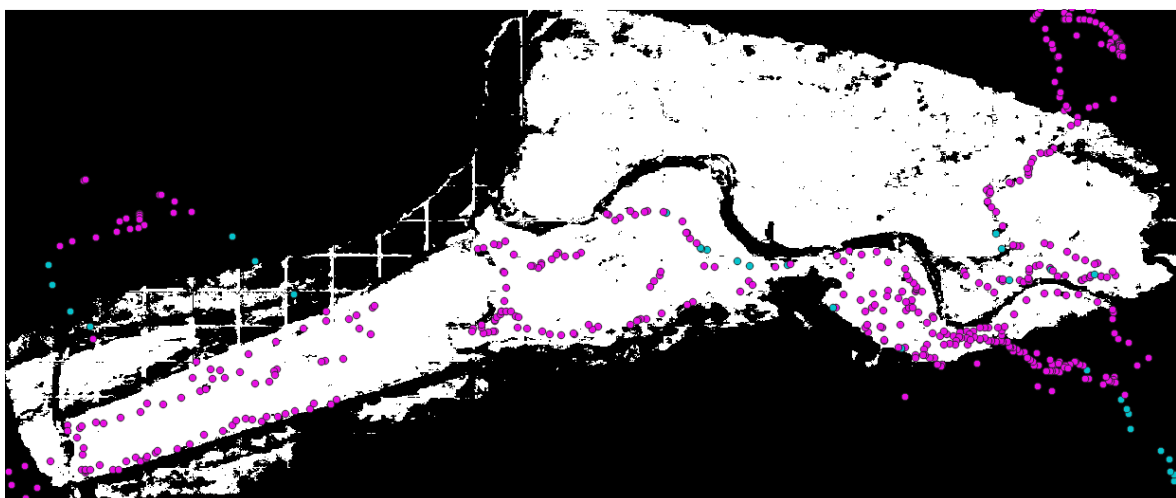


Figura 5.10: Máscara predicha por el modelo entrenado con las imágenes a menor resolución. Superpuestos, los puntos de *ground truth* que indican presencia (rosa) o ausencia (azul)

Si se observa la máscara predicha por este modelo existen dos cosas que destacan visualmente. Por un lado, las líneas verticales y horizontales de color blanco que aparecen en el centro-izquierda de la imagen. Realmente en esa zona no deberían aparecer artefactos de color blanco ya que no existen regiones de interés. Sin embargo el modelo está produciéndolos, en parte por que quizá no ha aprendido lo suficiente y en parte debido a la baja resolución de la imagen. Al tener una baja resolución, el tamaño del ortomosaico se ve disminuido. Y tal y como está implementado el proceso de inferencia, al igual que para el entrenamiento, el ortomosaico se divide en *patches* de un tamaño fijo (512x512) y se lleva a cabo la predicción para cada uno. Como cada fragmento ahora es de un mayor tamaño debido a la nueva resolución, pueden verse los cuadrados en la predicción resultante, fruto de juntar los distintos *patches*.

Valor real	Positivo real	294	15
	Negativo real	28	2
		Positivo predicho	Negativo predicho
		Predicción	

Figura 5.11: Matriz de confusión del modelo entrenado a partir del conjunto de imágenes con menor resolución

Por otro lado, destaca la elevada densidad de regiones de interés, existiendo muy pocos artefactos o 'islas' entre regiones señaladas como *N. noltei*. La mayoría de estas regiones se concentran en zonas muy homogéneas y consistentes. Aunque esto puede parecer algo positivo, en este caso no lo es. Las métricas previas (alto *recall* pero baja *precision*) indican la tendencia del modelo a sobresegmentar regiones de interés generando un elevado número de falsos positivos. Y esto puede corroborarse a través de la métrica de validación 4.5, donde esta máscara predicha recibe un porcentaje de acierto en base a los puntos de *ground truth* del 87.32 %. Si se desglosa esta métrica haciendo uso de la matriz de confusión de la Figura 5.11 se observa como el modelo acierta un total 294 puntos positivos mientras que clasifica erróneamente 28 puntos que realmente son negativos. En cuanto al acierto de puntos negativos, el modelo únicamente acierta 2, clasificando erróneamente 15 que realmente son positivos.

5.4.3.- Discusión de los resultados

A diferencia del experimento previo, en el que los resultados de ambos enfoques fueron muy similares tanto en las métricas preestablecidas como en la métrica personalizada de validación basada en los puntos de *ground truth*, en este experimento se observa una diferencia notable entre los resultados obtenidos. Esta discrepancia sugiere que la resolución de las imágenes juega un papel determinante en el rendimiento del modelo. A continuación, se analiza en detalle esta diferencia.

En cuanto a las métricas empleadas durante el *pipeline* de entrenamiento del modelo (error, *precision*, *recall* y porcentaje de acierto a nivel de píxel), el modelo entrenado con ortomosaicos de mayor resolución obtiene mejores resultados en todas ellas, excepto en *recall*. Sin embargo, esta diferencia en *recall* es mínima y, además, el modelo de mayor resolución logra un equilibrio entre *precision* y *recall*. A diferencia del modelo entrenado con ortomosaicos de menor resolución, no presenta un sesgo excesivo ni hacia la omisión ni hacia la sobreestimación de las áreas de interés. Esto es positivo, ya que implica que el modelo no solo identifica correctamente las zonas relevantes, sino que también minimiza la cantidad de falsos positivos y falsos negativos.

Este comportamiento no es solo teórico, sino que queda respaldado por las matrices de confusión presentadas en las Figuras 5.8 y 5.11. La primera, correspondiente al modelo entrenado con ortomosaicos de mayor resolución, muestra un mayor número de aciertos tanto en los puntos positivos como en los negativos, lo que se traduce en una menor generación de falsos positivos y falsos negativos. Esto se refleja en el porcentaje de acierto, donde este modelo alcanza un 92.04 %, frente al 87.32 % obtenido por el modelo entrenado con ortomosaicos de menor resolución.

A todo esto hay que sumarle la aparición de artefactos en la predicción, los cuales se vuelven más evidentes a medida que la resolución disminuye. Como se ha mencionado anteriormente, al trabajar con imágenes de menor resolución, el tamaño del ortomosaico se reduce y, dado que el proceso de inferencia sigue el mismo esquema de división en *patches* que el entrenamiento, el modelo genera predicciones con patrones cuadriculados más notorios al reconstruir la máscara. En contraste, al aumentar la resolución, estos efectos se atenúan, lo que no solo mejora la calidad visual de la segmentación, sino que también contribuye a una detección más uniforme de las áreas de interés.

Por consiguiente, queda evidenciado que el uso de imágenes de mayor resolución permite al modelo realizar una detección más precisa y equilibrada de las áreas de interés. Esto sugiere que, en un escenario práctico, capturar imágenes a una mayor resolución podría ser preferible para maximizar la fiabilidad de las predicciones. No obstante, esta elección debe valorarse

frente a las implicaciones operativas, dado que volar el dron a una menor altura conlleva un mayor tiempo de captura y procesamiento de datos. En este sentido, los resultados obtenidos proporcionan un marco de referencia útil para tomar decisiones informadas sobre la planificación de la adquisición de imágenes, permitiendo encontrar un equilibrio entre precisión del modelo y eficiencia operativa.

6. Conclusiones y trabajos futuros

El presente capítulo recopila las conclusiones derivadas del estudio realizado y plantea posibles líneas de trabajo futuras. En primer lugar, se expondrán las observaciones extraídas a partir de la realización de este estudio. Posteriormente, se discutirán mejoras y extensiones que podrían explorarse en futuras investigaciones para optimizar y ampliar el alcance del trabajo desarrollado.

6.1.- Conclusiones

El presente trabajo surge ante la necesidad de mejorar la eficiencia y precisión en la monitorización de praderas marinas, en particular la *Nanozostera noltei*, especie clave en la conservación de los ecosistemas costeros y sujeta a diversas figuras de protección. La segmentación manual de imágenes aéreas, actualmente empleada por el Centro Oceanográfico de Gijón, resulta limitada en cuanto a escalabilidad y consumo de recursos. En este contexto, el empleo de redes neuronales profundas se perfila como una solución eficaz para automatizar el proceso de segmentación, aliviando la carga operativa asociada al enfoque manual y mejorando la reproducibilidad de los resultados. De este modo, este trabajo constituye una aportación puntual dentro de un propósito más general por aplicar técnicas de inteligencia artificial a la gestión ambiental, evaluando su viabilidad para el análisis preciso y eficiente de imágenes capturadas mediante drones en entornos naturales sujetos a protección.

En esta memoria se ha abordado de manera integral el preprocesamiento y análisis de un conjunto de ortomosaicos adquiridos por un dron, así como la configuración del entrenamiento de un modelo de segmentación. Desde la introducción, donde se destacó la importancia de la monitorización de la *N. noltei* y las ventajas de automatizar este proceso, hasta los experimentos realizados, se han evaluado y discutido diversas metodologías y enfoques relacionados con los datos empleados, con el objetivo final de mejorar la calidad y precisión de la segmentación de áreas de interés en el terreno. A lo largo de la memoria, se han detallado los pasos que van desde el tratamiento de los datos para su correcta disposición hasta la ejecución de los experimentos, presentando los resultados obtenidos y las implicaciones de estos en términos de precisión, eficiencia y viabilidad operativa.

En primer lugar, se ha subrayado la importancia de contar con un conjunto de datos lo más robusto y preciso posible para entrenar eficazmente un modelo de segmentación. Para ello, ha sido fundamental realizar un procesamiento intensivo de los datos antes de la fase de entrenamiento. El eje principal de este preprocesamiento ha sido la construcción de las máscaras

binarias en formato imagen. De hecho, las máscaras *TIFF* resultantes del proceso fueron de una calidad excepcional, idénticas a las proporcionadas inicialmente en formato *GeoJSON*, eliminando eficazmente los artefactos generados durante el proceso debido a la naturaleza de los datos originales. De este modo, se obtuvo un conjunto de datos listo para su empleo, que refleja fielmente los datos originales proporcionados por el Centro Oceanográfico de Gijón. La calidad del conjunto de datos está directamente relacionada con el rendimiento del modelo en la segmentación, lo que ha hecho de este paso un elemento clave dentro de este proyecto.

Las pruebas principales se han centrado en evaluar el comportamiento de un mismo modelo bajo diferentes configuraciones del conjunto de datos. En una primera aproximación, se comparó el rendimiento del modelo utilizando la totalidad de los *patches* disponibles frente a una versión filtrada del conjunto, en la que se eliminaron aquellos que no contenían información relevante. Esta estrategia de filtrado permitió reducir significativamente el volumen de datos y los tiempos de entrenamiento, sin empeorar la coherencia en la segmentación de las regiones de interés. No obstante, este enfoque también implica ciertos riesgos, como la posibilidad de incrementar las predicciones erróneas en regiones donde la información es ambigua o escasa, lo que pone de manifiesto la necesidad de encontrar un equilibrio adecuado entre la cantidad y la calidad de los datos empleados en el entrenamiento.

El segundo experimento abordó el impacto de la resolución de las imágenes en el rendimiento del modelo. Los resultados evidenciaron cómo una mayor resolución mejora notablemente la capacidad del modelo para identificar y segmentar con precisión las áreas de interés, aunque a costa de un incremento significativo en la demanda computacional y en el tiempo necesario para el procesamiento y entrenamiento. Este análisis puso de manifiesto la importancia de encontrar un equilibrio entre la precisión del modelo y los recursos disponibles. En contextos donde la eficiencia y la escalabilidad son factores clave, la elección de la resolución adecuada debe ser cuidadosamente considerada, ya que condiciona tanto la calidad del modelo y su coste computacional asociado, como la viabilidad práctica de la adquisición de datos. De esta manera, el experimento no solo permitió evaluar el desempeño del modelo, sino que también aportó criterios concretos para la toma de decisiones en futuras misiones de adquisición de imágenes con drones, estableciendo un marco más claro para optimizar el compromiso entre resolución y coste operativo.

Así, este trabajo se enmarca como parte de un esfuerzo más amplio por mejorar la monitorización de ecosistemas marinos, como las praderas de *Nanozostera noltei*, cuya preservación es vital tanto para la biodiversidad local como para los servicios ecosistémicos que proporcionan. La colaboración entre el Grupo de Ecología de Angiospermas Marinas del IEO-CSIC y el área de *Inteligencia Artificial y Ciencias de la Computación* de la Universidad de Oviedo ha proporcionado una base sólida para avanzar hacia la automatización del proceso de

segmentación de imágenes aéreas. En este contexto, el uso de redes neuronales profundas ha demostrado ser una alternativa eficaz frente al enfoque manual tradicional, especialmente en tareas de seguimiento de grandes superficies como las praderas de la ría de Villaviciosa. De cara a futuros trabajos, este estudio sienta las bases para el desarrollo de herramientas más avanzadas que optimicen los procesos de adquisición de imágenes, al tiempo que abre la puerta a la exploración de nuevos modelos y métodos capaces de mejorar el rendimiento y la precisión de la segmentación. La implementación de enfoques más eficientes y escalables no solo permitirá ampliar el alcance de los proyectos de monitoreo, sino que también contribuirá a una gestión más sostenible de estos ecosistemas, en línea con los actuales esfuerzos de conservación.

6.2.- Trabajos futuros

A partir de las conclusiones extraídas, en esta sección se proponen posibles líneas de trabajo que podrían mejorar o ampliar los resultados alcanzados. Se plantearán optimizaciones, ajustes metodológicos y nuevas direcciones de investigación que podrían ser exploradas en estudios posteriores.

6.2.1.- Empleo de una versión refinada de las máscaras

Como se mencionó en el capítulo 4, las máscaras actuales son aproximadas y no detallan con precisión las regiones de presencia de *N. noltei* a nivel de píxel. No obstante, se está desarrollando una nueva versión de estas máscaras con una delimitación más precisa de las zonas de interés. Esta busca reducir la incertidumbre generada por los bordes difusos en las máscaras actuales y mejorar la correspondencia entre las regiones de pradera marina y su representación en las imágenes aéreas.

Hubiera sido interesante contar con esta versión desde el inicio del trabajo para evaluar el impacto de unos datos más refinados en el rendimiento del modelo. No obstante, al no haber estado disponible a tiempo, los experimentos se han llevado a cabo con las máscaras iniciales con el objetivo de tener un primer marco de referencia que permitiera validar la viabilidad del enfoque propuesto. Aunque los resultados obtenidos están condicionados por la calidad de las máscaras disponibles, este estudio preliminar resulta fundamental para determinar si la metodología es prometedora y justifica futuras investigaciones con datos más precisos. De este modo, cuando las máscaras refinadas estén disponibles, se dispondrá ya de un marco de trabajo que facilitará su integración y evaluación, acelerando el proceso hacia un modelo más robusto y preciso en la delimitación de las praderas de *N. noltei*.

Un posible experimento futuro consistiría en realizar un análisis comparativo entre ambas versiones, midiendo su efecto en la precisión de las predicciones y en la generalización del modelo. Esto ayudaría a evaluar hasta qué punto una segmentación manual más precisa y que conlleva más tiempo puede influir en la calidad de las predicciones y en la estabilidad del modelo.

6.2.2.- Empleo de imágenes multiespectrales

Otra posible línea de investigación futura podría ir ligada al uso de las 7 bandas disponibles en algunas imágenes y explorar más allá de las típicas *RGB*. El problema que existe con relación a este tema es que solo se disponen de 4 imágenes con estas características actualmente. Esto requeriría que el equipo del Centro Oceanográfico de Gijón volase el dron de nuevo sobre la ría pero con la configuración pertinente para obtener imágenes con bandas

espectrales lo que supone un coste añadido, por un lado en el personal encargado de volar el dron y etiquetar esas nuevas imágenes y por otro en tiempo y recursos ya que volar el dron con dicha configuración hace que la toma de las imágenes sea más lenta.

En otras investigaciones centradas en la monitorización de *Posidonia oceanica* [25] [26] se ha observado que el empleo de imágenes multiespectrales de alta resolución ha mostrado ser útil para la discriminación de hábitats marinos, aunque con ciertas limitaciones en temas de precisión. Si bien las imágenes multiespectrales pueden proporcionar resultados razonables para clasificar comunidades en áreas no tan profundas, su resolución espacial y espectral puede restringir su aplicación en áreas más profundas o con escasa transparencia, donde la reflectancia y la radiación se ven absorbidas por el agua [25].

La planta estudiada en este trabajo es *N. noltei* en lugar de *P. oceanica*. La principal diferencia es que la *N. noltei* se fotografía en marea baja, es decir, cuando la planta queda al descubierto, por lo que el problema de absorción de la radiación por el agua y la dependencia de la transparencia de esta, comentado por los autores del artículo [25] no afectaría en este caso. Además este hecho hace que se pueda diferenciar mejor la planta de interés de las algas debido a que estas últimas se secan cuando les da el sol y por tanto cambian de color. Incluso se aprecian mayores diferencias dentro de determinadas bandas espectrales como se vio en el estudio [27], lo que facilita la identificación de la planta de interés.

Por lo tanto, sería interesante estudiar si el empleo de bandas espectrales destaca la presencia de las praderas de *N. noltei*, aportando información relevante al modelo que hace que aprenda a segmentar las regiones de interés de una forma mucho más clara y precisa en comparación con el modelo actual entrenado solo con imágenes *RGB*. De ser así, habría que estudiar la mejoría obtenida y valorar si merece la pena volar el dron con la configuración para obtener más imágenes con 7 bandas o si por el contrario el modelo realmente no es tan superior o no aporta una precisión extra que sea tan relevante como para asumir el coste añadido y los inconvenientes que presenta la toma de este tipo de imágenes.

Bibliografía

- [1] J. Ruiz, E. Guillén, A. Ramos Segura, and M. Otero, *Atlas de las praderas marinas de España*. Murcia-Alicante-Málaga: IEO/IEL/UICN, 2015.
- [2] M. González Olmedo, *Bayesian Plug Play Methods for Inverse Problems in Imaging*. PhD thesis, Universidad de la República de Uruguay, 12 2021.
- [3] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” 2015.
- [4] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” 2015.
- [5] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Encoder-decoder with atrous separable convolution for semantic image segmentation,” 2018.
- [6] Y. Gonzalez-Cid, F. Bonin-Font, E. G. Font, A. M. Torres, M. M. Abadal, G. O. Codina, H. Hinz, L. P. Briones, and F. Tomas, “Autonomous marine vehicles and cnn: Tech tools for posidonia meadows monitoring,” in *OCEANS 2021: San Diego – Porto*, pp. 1–8, 2021.
- [7] M. Martin-Abadal, E. Guerrero-Font, F. Bonin-Font, and Y. Gonzalez-Cid, “Deep semantic segmentation in an auv for online posidonia oceanica meadows identification,” *IEEE Access*, vol. 6, pp. 60956–60967, 2018.
- [8] F. Bonin-Font, A. Martorell-Torres, M. M. Abadal, C. Muntaner-González, B. M. Nordfeldt-Fiol, Y. González-Cid, G. Oliver-Codina, J. Máñez-Crespo, X. Reynés, L. Pereda, G. Hernan, and F. Tomás, “Measuring the temporal evolution of seagrass posidonia oceanica coverage using autonomous marine robots and deep learning,” *Estuarine, Coastal and Shelf Science*, vol. 312, p. 109029, 2025.
- [9] À. Giménez-Romero, D. Ferchichi, P. Moreno-Spiegelberg, T. Sintes, and M. A. Matías, “Mapping the distribution of seagrass meadows from space with deep convolutional neural networks,” *bioRxiv*, 2024.
- [10] A. Inc., “What is a tiff file?,” n.d. Accessed: January 21, 2025.
- [11] P. Aero, “What are geotiff files and why are they so useful in drone mapping?,” n.d. Accessed: January 21, 2025.
- [12] M. Yushchuk, “Preparing aerial imagery for crop classification,” 2019. Accessed: January 21, 2025.

- [13] H. Butler, M. Daly, A. Doyle, S. Gillies, T. Schaub, and S. Hagen, “The GeoJSON Format.” RFC 7946, Aug. 2016.
- [14] QGIS Development Team, *QGIS Geographic Information System*. QGIS Association,
- [15] Mapbox, “Rasterio documentation (stable version),” n.d. Accessed: January 21, 2025.
- [16] S. Developers, “Shapely documentation (stable version),” n.d. Accessed: January 21, 2025.
- [17] K. Jordahl, J. V. den Bossche, M. Fleischmann, J. Wasserman, J. McBride, J. Gerard, J. Tratner, M. Perry, A. G. Badaracco, C. Farmer, G. A. Hjelle, A. D. Snow, M. Cochran, S. Gillies, L. Culbertson, M. Bartos, N. Eubank, maxalbert, A. Bilogur, S. Rey, C. Ren, D. Arribas-Bel, L. Wasser, L. J. Wolf, M. Journois, J. Wilson, A. Greenhall, C. Holdgraf, Filipe, and F. Leblanc, “geopandas/geopandas: v0.8.1,” July 2020.
- [18] Dovahcrow, “patchify.py,” n.d. Accessed: January 21, 2025.
- [19] F. Chollet *et al.*, “Keras.” <https://github.com/fchollet/keras>, 2015.
- [20] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. Software available from tensorflow.org.
- [21] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [22] L. P. Soares, “Binary mask for image processing,” 2019. Accessed: January 21, 2025.
- [23] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” 2019.
- [24] K. Team, “Oxford pets image segmentation with keras,” 2025. Accessed: January 23, 2025.
- [25] S. F. Rende, A. Bosman, R. D. Mento, F. Bruno, A. Lagudi, A. D. Irving, L. Dattola, L. D. Giambattista, P. Lanera, R. Proietti, L. Parlagreco, M. Stroobant, and E. Cellini, “Ultra-high-resolution mapping of posidonia oceanica (l.) delile meadows through acoustic, optical data and object-based image classification,” *Journal of Marine Science and Engineering*, 2020.

- [26] R. Matarrese, M. Acquaro, A. Morea, K. Tijani, and M. Chiaradia, “Applications of remote sensing techniques for mapping posidonia oceanica meadows,” pp. IV – 906, 08 2008.
- [27] S. Oiry, B. Laurent, B. Davies, A. Sousa, P. Rosa, and P. Gernez, “Remote sensing discrimination of seagrass and green macroalgae : hyperspectral library and drone-mounted multispectral camera,” 08 2023.

A. Anexo I: Ajuste de hiperparámetros

El ajuste de hiperparámetros del modelo se presenta en este anexo como un paso complementario a los experimentos previos, centrados principalmente en la selección y disposición de los datos de entrada. Esta fase se llevó a cabo una vez finalizados dichos experimentos, utilizando la configuración de datos que ofreció mejores resultados, con el objetivo de explorar si era posible mejorar aún más el rendimiento del modelo. Durante los experimentos se empleó una configuración estándar de hiperparámetros para todos los casos, garantizando así una comparación justa entre ellos. Ajustar los hiperparámetros en cada configuración habría supuesto un coste computacional y de tiempo elevado. Además, dado que el enfoque principal del trabajo gira en torno a los datos, y que el impacto del ajuste sobre el rendimiento, como se verá a continuación, no fue especialmente significativo, se ha optado por incluir este análisis como un anexo. Aun así, se considera relevante conservarlo, ya que permite identificar qué hiperparámetros influyen más en el proceso de entrenamiento y podría ser útil en futuros trabajos.

Para ello, se utilizó el *framework* de optimización Optuna, el cual permite explorar de manera eficiente distintas configuraciones de hiperparámetros mediante estrategias de búsqueda automatizadas. En este caso, la optimización se formuló como un problema de minimización de la función de pérdida en el conjunto de validación. Se definió un espacio de búsqueda que abarcaba hiperparámetros como el *learning rate*, el tamaño del *batch*, los criterios de reducción del ritmo de aprendizaje (*reduce_lr_factor*) y los parámetros de *early stopping* (*early_stopping_patience*, *reduce_lr_patience*).

Se definieron 30 iteraciones en el proceso de optimización, durante las cuales Optuna generó una nueva combinación de valores para los hiperparámetros dentro de los rangos establecidos que pueden verse en la tabla A.1. Gracias a la implementación de *checkpoints*, al finalizar el proceso se obtuvo y almacenó la mejor versión del modelo. Tras completar las iteraciones, se seleccionó la configuración de hiperparámetros que resultó en el menor valor de la función de pérdida en validación. El resultado puede consultarse en la tabla A.2.

Hiperparámetro	Rango de búsqueda
Tasa de aprendizaje (<i>learning rate</i>)	$[10^{-5}, 10^{-2}]$
Tamaño del batch (<i>batch size</i>)	[4, 32]
Factor de reducción de <i>ReduceLROnPlateau</i>	[0.05, 0.5]
Paciencia del <i>early stopping</i>	[10, 50]
Paciencia de <i>ReduceLROnPlateau</i>	[5, 20]

Tabla A.1: Rangos de búsqueda de los hiperparámetros

Hiperparámetro	Valor
Tasa de aprendizaje (<i>learning rate</i>)	0.000124
Tamaño del batch (<i>batch size</i>)	16
Factor de reducción de <i>ReduceLROnPlateau</i>	0.35
Paciencia del <i>early stopping</i>	27
Paciencia de <i>ReduceLROnPlateau</i>	9
Valor del error en validación	0.11144

Tabla A.2: Hiperparámetros óptimos

Como se mencionó en la introducción, el ajuste de hiperparámetros no tuvo un impacto significativo en el rendimiento del modelo en comparación con la configuración utilizada en los experimentos previos. La diferencia en el error de validación es prácticamente inexistente. De hecho, el menor error obtenido en validación durante los experimentos se alcanzó utilizando la misma configuración de imágenes con la que se realizó el ajuste (conjunto filtrado y de mayor resolución). En este caso, el valor del error fue de 0.11159, lo que representa una diferencia de solo 0.00015 respecto al valor del error mostrado en la tabla A.2, una variación que es insignificante.

Este resultado pone de manifiesto que el ajuste de hiperparámetros no siempre es un paso imprescindible en el entrenamiento de un modelo de *deep learning*. En este caso, la configuración inicial de los hiperparámetros seleccionada para los experimentos fue elegida mediante un proceso preliminar de prueba y error, utilizando entre dos o tres de los valores más comunes para los hiperparámetros críticos, como el tamaño del lote y la tasa de aprendizaje. El resto de los hiperparámetros fueron seleccionados de manera más intuitiva, generalmente manteniendo su valor por defecto. Sin embargo, al comparar esta configuración inicial con la obtenida a través del ajuste de hiperparámetros, se observa que las diferencias son mínimas, lo que explica la escasa variación en el error obtenido. Esto puede deberse a que los valores

de estos parámetros, comúnmente establecidos en el entrenamiento de este tipo de modelos, suelen ser los que mejor funcionan de manera generalizada, sin necesidad de realizar ajustes específicos para cada caso. En muchos casos, estos valores estándar han sido ampliamente validados en diversas aplicaciones, lo que hace que, independientemente de la tarea, proporcionen un rendimiento satisfactorio sin requerir pruebas exhaustivas para cada configuración particular.

La figura A.1 obtenida del *dashboard* de Optuna muestra la evolución del error de validación (eje Y) a lo largo de las distintas iteraciones o *trials* (eje X) realizadas durante el proceso de ajuste de hiperparámetros. En cada iteración, se evalúa una combinación distinta de hiperparámetros, registrando el valor del error correspondiente al modelo entrenado con dicha configuración.

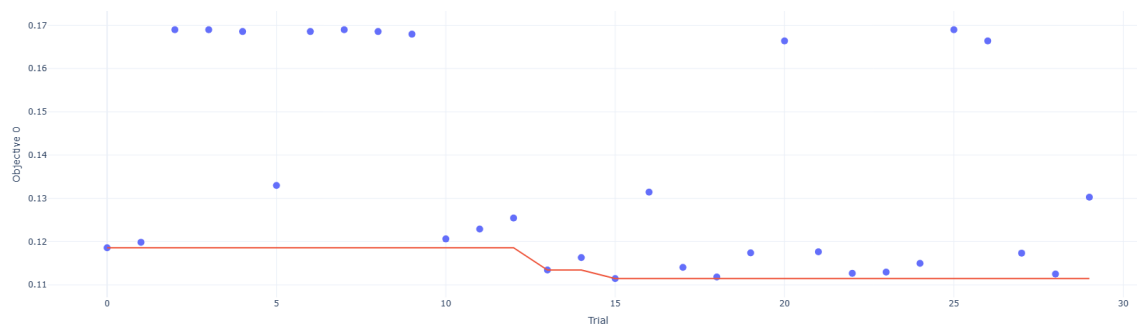


Figura A.1: Fluctuación del error de validación durante las distintas iteraciones.

Como se puede observar, las primeras iteraciones presentan una alta variabilidad en los resultados, con errores de validación que en muchos casos superan el 0.16. Esto indica que las combinaciones iniciales de hiperparámetros exploradas no eran especialmente prometedoras. Sin embargo, a partir del *trial* número 10 aproximadamente, se empieza a observar una tendencia descendente en los errores obtenidos, reflejando una mejora progresiva en la selección de hiperparámetros. Esta tendencia se ve reforzada por la línea naranja, que representa el mejor valor alcanzado hasta ese punto de la búsqueda, el cual disminuye de forma escalonada conforme Optuna encuentra configuraciones más óptimas.

El mínimo valor de error alcanzado durante las 30 iteraciones realizadas se sitúa ligeramente por encima de 0.11, y aparece en varias ocasiones entre las iteraciones 15 y 29. Esta estabilización sugiere que el optimizador ha identificado una región prometedora del espacio de hiperparámetros, donde pequeñas variaciones producen errores similares y cercanos al mínimo observado. No obstante, también se pueden apreciar algunas oscilaciones en los últimos *trials*, lo cual es característico del enfoque exploratorio de Optuna, que sigue evaluando nuevas combinaciones aunque ya haya encontrado una solución competitiva.

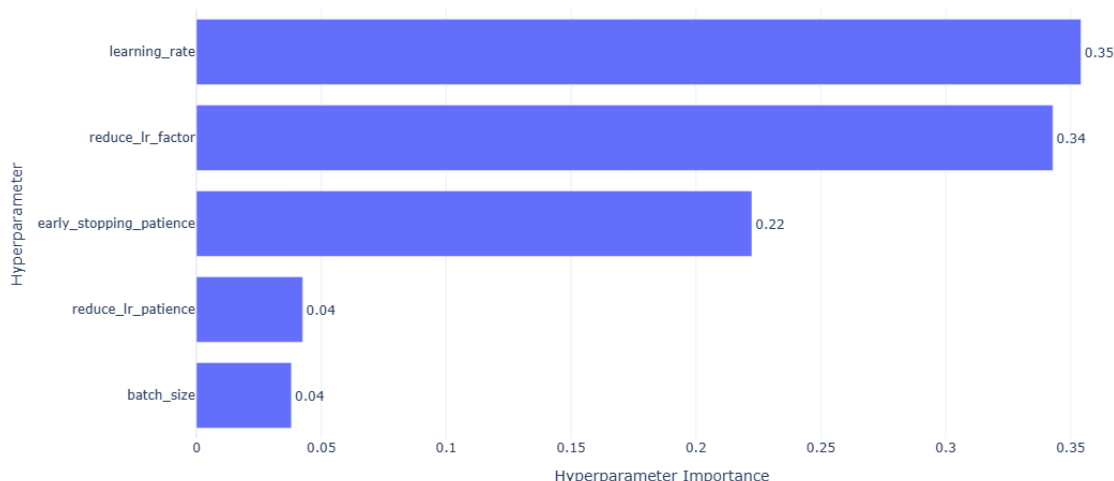


Figura A.2: Importancia asignada por Optuna a cada hiperparámetro en relación con su contribución a la reducción del error de validación.

La figura A.2 obtenida del *dashboard* de Optuna muestra la importancia relativa de cada uno de los hiperparámetros considerados durante el proceso de optimización. Esta información se obtiene mediante un análisis de sensibilidad que estima cuánto contribuye cada hiperparámetro a la variación en el error de validación, manteniendo constantes los demás parámetros.

Se observa que los hiperparámetros más determinantes para el rendimiento del modelo son el *learning rate* y el factor de reducción del *learning rate*, con una importancia relativa del 35 % y 34 %, respectivamente. Esto sugiere que el ajuste de estos dos valores tiene un impacto significativo en la capacidad del modelo para minimizar el error de validación. El hecho de que ambos estén relacionados con la tasa de aprendizaje refuerza la idea de que una correcta gestión del ritmo de actualización de pesos durante el entrenamiento es clave para obtener buenos resultados en este caso.

El hiperparámetro relativo a la paciencia del *early stopping* también presenta una relevancia considerable (22 %), lo que indica que la estrategia de parada temprana influye de forma notable en el rendimiento final, probablemente porque evita el sobreajuste en configuraciones con muchas épocas. En contraste, los hiperparámetros relativos al tamaño del lote y la paciencia de reducción del valor del *learning rate* apenas contribuyen a la mejora del modelo, con una importancia estimada de tan solo 4 % cada uno.

Como se tenía el mejor modelo tras el proceso completo de optimización, se decidió utilizarlo para hacer inferencia sobre el ortomosaico que se viene utilizando para validación con el objetivo de contradecir o reafirmar la hipótesis de que este ajuste hiperparamétrico no ha supuesto una mejora significativa. El resultado de la inferencia puede verse en la figura A.3.

A simple vista esta máscara a penas difiere de la que se obtuvo con el mejor modelo empleado en los experimentos, el modelo entrenado a partir de datos filtrados de mayor resolución.

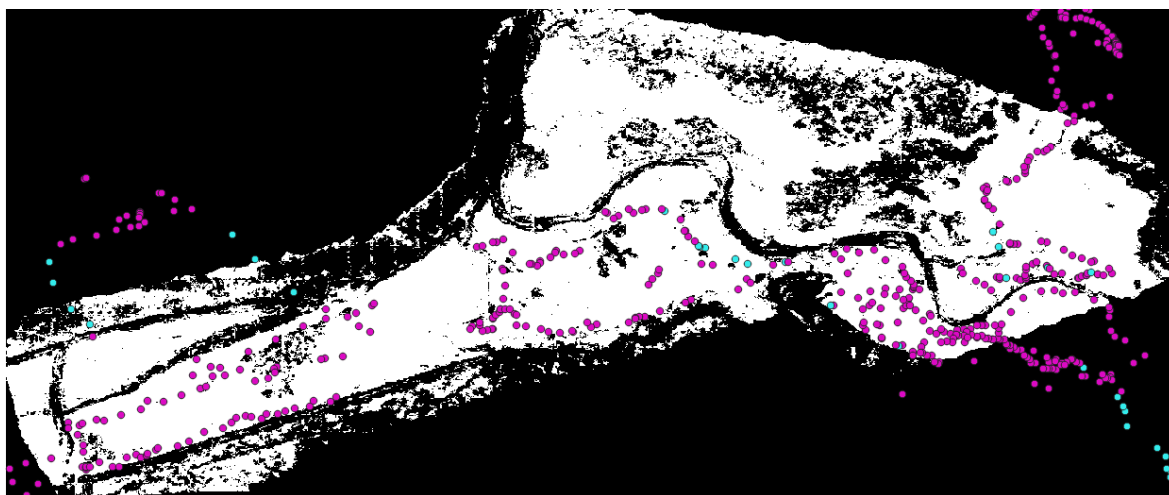


Figura A.3: Máscara predicha por el modelo con los mejores hiperparámetros tras la optimización.

Pero como esta es una evaluación subjetiva, una forma de conseguir compararlas de manera más objetiva es emplear la métrica de validación, en la que este modelo ha conseguido acertar un 91.74 % del total de puntos de *ground truth*, mínima diferencia respecto al otro modelo que consiguió un porcentaje de acierto del 92.02 %. De hecho, si se calcula la matriz de confusión, presentada en la figura A.4, es idéntica a la del mejor modelo de los experimentos de secciones anteriores, con la única diferencia de que esta acierta un punto positivo menos, que lo clasifica erróneamente como negativo. Salvando esta discrepancia, ambos modelos aciertan los mismos puntos.

Por tanto, se confirma que el ajuste de hiperparámetros no ha supuesto una mejora significativa en el rendimiento del modelo, ni a nivel cuantitativo ni cualitativo. Las métricas obtenidas y las predicciones generadas sobre el ortomosaico de validación refuerzan la idea de que la configuración inicial utilizada en los experimentos principales ya ofrecía un comportamiento muy cercano al óptimo. Este resultado pone de manifiesto la eficacia de ciertos valores de hiperparámetros ampliamente aceptados en la literatura, que tienden a funcionar adecuadamente de forma generalizada. En el contexto global del trabajo, puede concluirse que la calidad y el diseño del conjunto de datos han sido factores más determinantes que el ajuste de los hiperparámetros del modelo.

Valor real	Positivo real	309	15
	Negativo real	13	2
		Positivo predicho	Negativo predicho
		Predicción	

Figura A.4: Matriz de confusión del modelo con los mejores hiperparámetros tras la optimización.